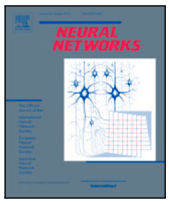
Neural Networks 187 (2025) 107400

Contents lists available at ScienceDirect

Neural Networks

journal homepage: www.elsevier.com/locate/neunet



Full Length Article

A lightweight All-MLP time–frequency anomaly detection for IIoT time series

Lei Chen [a,*], Xinzhe Cao [b], Tingqin He [b], Yepeng Xu [a], Xuxin Liu [a], Bowen hu [a]

[a] School of Information and Electrical Engineering, Hunan University of Science and Technology, Xiangtan, 411201, China
[b] School of Computer Science and Engineering, Hunan University of Science and Technology, Xiangtan, 411201, China

ARTICLE INFO

Keywords:
Industrial Internet of Things (IIoT)
Time series
Anomaly detection
Time–frequency joint learning
All-MLP architecture
Lightweight network

ABSTRACT

Anomaly detection in the Industrial Internet of Things (IIoT) aims at identifying abnormal sensor signals to ensure industrial production safety. However, most existing models only focus on high accuracy by building a bulky neural network with deep structures and huge parameters. In this case, these models usually exhibit poor timeliness and high resource consumption, which makes these models unsuitable for resource-limited edge industrial scenarios. To solve this problem, a lightweight All-MLP time–frequency anomaly detection model is proposed for IIoT time series, namely LTFAD. *Firstly*, unlike traditional deep and bulky solutions, a shallow and lightweight All-MLP architecture is designed to achieve high timeliness and low resource consumption. *Secondly*, based on the lightweight architecture, a dual-branch network is constructed to improve model accuracy by simultaneously learning "global to local" and "local to global" reconstruction. *Finally*, time–frequency joint learning is employed in each reconstruction branch to further enhance accuracy. To the best of our knowledge, this is the first work to develop a time–frequency anomaly detection model based only on the shallow All-MLP architecture. Extensive experiments demonstrate that LTFAD can quickly and accurately identify anomalies on resource-limited edge devices, such as the Raspberry Pi 4b and Jetson Xavier NX. The source code for LTFAD is available at https://github.com/infogroup502/LTFAD.

1. Introduction

The Industrial Internet of Things (IIoT) plays a crucial role in industrial intelligence and smart manufacturing. Anomaly detection is critical for IIoT to identify abnormal sensor signals and ensure industrial production safety (Yu et al., 2024). To uncover abnormal events in IIoT, early methods (Li, Izakian, Pedrycz, & Jamal, 2021) rely on statistical features of sensor signals, such as distance, variance, and density, to build detection models. However, these models often exhibit limited accuracy. To solve this problem, machine learning-based (Langfu et al., 2023) and deep learning-based models (Duan et al., 2022) have been explored. They use large datasets and expensive artificial labels to improve model accuracy through supervised learning. However, anomaly labels are very scarce in IIoT systems, which makes supervised-based or semi-supervised-based deep anomaly detection models gradually lose competitiveness. In this case, unsupervised deep learning-based anomaly detection models (Zhu, Li, Dorsey, & Luo, 2023) have become mainstream in recent research.

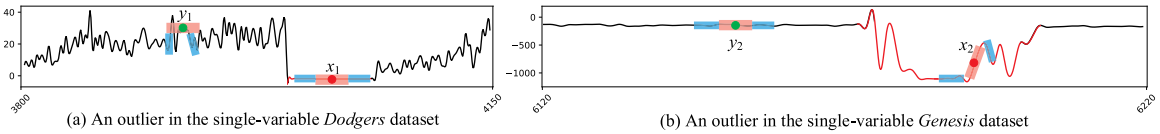
With the advent of Big Data and Industry 4.0, IIoT time series have new characteristics and also bring some new challenges for anomaly detection (Sgueglia, Di Sorbo, Visaggio, & Canfora, 2022).

1.1. Challenges

- *Accurate feature extraction*. Driven by the growing demand for high-quality production, traditional production processes are becoming increasingly complex, showing diverse patterns such as seasonality, periodicity, and trends. These complex production processes require fine-grained intelligent management and monitoring. Thus, IIoT must deploy more sensors to monitor the production more intensively and comprehensively. In this case, the collected IIoT time series exhibit new characteristics: more complex and diverse features, coupled variables, larger volumes and fewer labels. These new characteristics complicate the task of distinguishing abnormal events from normal sequences. Therefore, an excellent IIoT anomaly detection model must have strong representation capabilities to accurately extract features from time series.
- *High timeliness and low consumption*. Typically, control devices in industrial scenarios have limited resources, such as meager CPU and GPU computing power and restricted storage capacity. However, fine-grained intelligent industrial control requires real-time and accurate management on these resource-limited devices. In this context, an effective time series anomaly detection (TSAD) model should possess high

* Corresponding author.
E-mail address: chenlei@hnust.edu.cn (L. Chen).

https://doi.org/10.1016/j.neunet.2025.107400
Received 5 November 2024; Received in revised form 3 February 2025; Accepted 12 March 2025
Available online 22 March 2025



(a) An outlier in the single-variable *Dodgers* dataset (b) An outlier in the single-variable *Genesis* dataset

Fig. 1. Inspirational ideas for identifying anomalies.

timeliness, low resource consumption and high accuracy. Among them, high timeliness and low resource consumption should prioritize to accuracy. Therefore, the ideal TSAD model should minimize computing and storage resource usage while detecting IIoT anomalies as quickly as possible. Unfortunately, existing models mainly focus on high accuracy, while less efforts have been made on the issue of limited resource.

1.2. Existing solutions

As for the first challenge, most existing solutions focus on building bulky models with deep structures and huge parameters to enhance feature capture capabilities. For example, the AnomalyTransformer model (Xu, Wu, Wang, & Long, 2022) and the DCdetector model (Yang, Zhang, Zhou, Wen, & Sun, 2023) employ transformer and attention networks as feature extractors, respectively. Tang, Xu, Yang, Tang, and Zhao (2023) uses the Gated Recurrent Unit network (GRU) to accurately capture both short-term and long-term temporal dependencies. The MSt-Gat model (Ding, Sun, & Zhao, 2023) and the 2P-DAs model (Chang, Chen, Su, Xie, & Li, 2024) use graph neural networks (GNN) and generative adversarial networks (GAN) to capture features more accurately. However, the huge parameters in these models often result in high CPU/GPU resource consumption, and the deep architectures significantly increase training and detection times.

As for the second challenge, existing models are mainly divided into two categories. *One category* employs parallel computing (Nizam, Zafar, Lv, Wang, & Hu, 2022) and federated learning (Truong et al., 2022) to accelerate the detection model, and enhance timeliness. Although this solution achieves high timeliness, it requires more computing and storage resources. In this case, it is very difficult to deploy these models on resource-limited edge devices in IIoT. *The other category* applies lightweight components to replace computationally intensive components in bulky models. In this case, these models can reduce the number of parameters, improve timeliness and optimize the resource consumption (Fan, Liu et al., 2023; Huang et al., 2024; Sivapalan, Nundy, Dev, Cardiff, & John, 2022). However, these models still have deep structures, and their advantages in timeliness and resource consumption are still not satisfactory. Thus, some All-MLP architecture-based lightweight models (Chen, Li, Arik, Yoder, & Pfister, 2023; Wang, Ruan et al., 2024) have been proposed for time series forecasting. These models consist of shallow structures and few parameters, such that they can offer high timeliness and low resource consumption. However, the issue of All-MLP architecture based lightweight model still remains open for IIoT anomaly detection.

1.3. New insights

This paper aims to design an All-MLP architecture-based lightweight IIoT anomaly detection model with high accuracy, high timeliness, and low resource consumption. To achieve this goal, two bottlenecks need to be addressed.

• *The first bottleneck is how to design a lightweight All-MLP architecture*. Compared to normal timestamps, outliers often exhibit significantly different association patterns between the local and global neighbors. In Fig. 1, two segments of anomalies on two real-world univariate datasets are depicted. In the figure, the red and blue rectangular areas represent the local and global neighbors of a given timestamp, respectively. It can be seen in the figure that the association pattern between local and global neighbors of x_1 differ significantly from that of y_1. It means that the association pattern between local and global neighbors can be leveraged to distinguish abnormal events from normal timestamps. Motivated by this insight, two reconstruction branches, which consist of two-layer MLPs, are constructed as the lightweight All-MLP architecture. The first branch uses the local neighbors of a timestamp to reconstruct its global neighbors. In contrast, the second branch uses the global neighbors to reconstruct the local neighbors. Finally, an anomaly score is generated based on the reconstruction errors of the two branches to identify anomalies.

• *The second bottleneck is how to enhance accuracy under the lightweight All-MLP architecture*. Although the lightweight All-MLP architecture improves model timeliness and reduces resource consumption, its feature extraction capability remains limited. Recently, some works (Xu, Zeng and Xu, 2024; Yi et al., 2024) have demonstrated that the features of time series are simpler and easier to extract in the frequency domain than that in the time domain. Therefore, we use time–frequency collaborative learning to enhance the feature extraction capability of the lightweight All-MLP architecture. Technically, we perform dual-branch reconstruction learning in both time and frequency domains simultaneously and align these two domains through the consistency of the reconstruction values.

1.4. Ours contributions

The main contributions of this paper are outlined as follows:

(1) A lightweight All-MLP time–frequency anomaly detection model, LTFAD, is proposed for IIoT time series, to achieve the performance of high accuracy, high timeliness and low resource consumption. To the best of our knowledge, this is the first work to develop a time–frequency anomaly detection model based only on the shallow All-MLP architecture.

(2) A novel dual-branch learning framework is designed by only using two-layer MLPs instead of traditional bulky neural networks. The designed framework can efficiently integrates "global-to-local" reconstruction, "local-to-global" reconstruction, and time–frequency joint learning.

(3) Extensive experiments are conducted on twelve IIoT time series and three IIoT edge devices: PC, Raspberry Pi 4b, and NVIDIA Jetson Xavier NX. The experimental results demonstrate that LTFAD outperforms several deep SOTA models in accuracy, timeliness and resource consumption. Specifically, LTFAD achieves first or second ranking on four metrics across five univariate and seven multivariate datasets, except the Recall score on Dodgers, the Precision score on SVDB, the Precision score on MSL and the Precision score on SWaT. Moreover, on the SVDB dataset, the Recall score of LTFAD is 15% higher than that of the second-ranked baseline. In addition, the detection time of LTFAD is 4–10 times faster than deep SOTA models.

2. Related work

2.1. Lightweight TSAD

Existing lightweight TSAD models can be divided into two categories.

The first category uses a machine learning or statistical model as the backbone, and employs another powerful model as a booster (Audibert, Michiardi, Guyard, Marti, & Zuluaga, 2022), to construct a hybrid high-performance model. In this category, the lightweight backbone

provides high timeliness and low resource consumption, while the powerful booster enhances model accuracy. For example, Langfu et al. (2023) integrates Fast-DTW and improved-KNN to achieve high performance. The DIFFI model (Carletti, Terzi, & Susto, 2023) optimizes the traditional Isolation Forest by combining global and local perspectives. The COUTA model (Xu, Wang, Jian, Liao and Wang, 2024) utilizes one-class SVM as the backbone and designs two calibration mechanisms as boosters. The LODA model (Pevnỳ, 2016) employs an ensemble learning mechanism to fuse multiple weak detectors into a strong one. Similarly, the DIF model (Xu, Pang, Wang, & Wang, 2023) uses a deep neural network to optimize the traditional Isolation Forest, thereby improving model accuracy. Although these models have advantages in speed and resource efficiency, their feature capture capability is limited. As a result, their accuracy drops severely when applied to complex IIoT time series.

The second category focuses on prune existing deep and bulky TSAD models or replacing computationally intensive components with lightweight alternatives, to construct a lightweight, high-performance model. In this category, deep neural network ensures high accuracy, while model compression and pruning optimize timeliness and resource consumption. For instance, the RTCAE model (Li et al., 2023) uses tensor decomposition to replace traditional convolution operations, and finally builds a lightweight convolutional autoencoder. The RG-GLD model (Zhou et al., 2024) employs a novel global–local distillation mechanism to optimize graph neural networks for efficient IoT anomaly detection. Similarly, the GAPCNN-SVM model (Gong et al., 2022) designs 1D-CNN to replace 2D-CNN. The LUAD model (Fan, Liu et al., 2023) uses the temporal convolutional network (TCN) to optimize the variational auto-encoder (VAE). The SimAD model (Zhong, Yu, Xi et al., 2024) compresses the Transformer network using a lightweight autoencoder architecture. Recently, lightweight MLP networks have been widely used to replace computationally sensitive components of bulky models, and finally construct lightweight TSAD models. For example, The ANNet model (Sivapalan et al., 2022) uses MLP to optimize LSTM cell. Similarly, the MEAformer (Huang et al., 2024) and PatchAD (Zhong, Yu, Yang, Wang and Yang, 2024) models employ MLP to replace the attention mechanism in Transformer networks. However, these models still have deep structures, limiting their improvements in timeliness and resource efficiency.

A recent innovation in lightweight design is the All-MLP solution for time series forecasting (Chen et al., 2023; Wang, Ruan et al., 2024). This solution utilizes only one-layer or two-layer MLPs with few parameters to perform time series forecasting. This design greatly enhances timeliness and reduces resource consumption. For example, the TSMixer model (Chen et al., 2023) designs a lightweight All-MLP architecture, and the TSP model (Wang, Ruan et al., 2024) introduces a computationally free recurrent mechanism based on MLP. Notably, despite their simplicity, these models achieve high accuracy, even surpassing several deep models. However, these models are currently limited to time series forecasting and cannot directly apply to time series anomaly detection.

2.2. Time–frequency-based TSAD

In recent years, some studies (Xu, Zeng et al., 2024; Yi et al., 2024; Zhang, Guo, Zhou, Zhang, & Lin, 2023) have highlighted a significant finding: complex features of some time series are difficult to extract in the time domain, but are simpler and easier to analyze in the frequency domain. Inspired by this finding, many time–frequency-based TSAD models have been proposed and achieved strong performance. Based on the use of frequency domain information, existing time–frequency-based TSAD models are mainly divided into two categories.

The first category treats frequency domain information as a complement to time domain data. For instance, the KfreqGAN model (Yao, Ma, & Ye, 2022) extracts frequency domain features and concatenates them with time domain data to generate augmented data. Based on this, the STFT-TCAN model (Tu, Liu, Yan, Jin, & Geng, 2024) integrates a TCN network with an attention mechanism to accurately capture the time–frequency features from the augmented data. Similarly, the FCVAE model (Wang, Pei et al., 2024) uses frequency domain information as the condition of a conditional variational autoencoder (CVAE) network to improve model accuracy. However, these models only use a small subset of frequency domain features such as amplitude and phase, and do not fully exploit the potential of the frequency domain.

The second category considers the time domain and frequency domain as two complementary branches to perform time–frequency collaborative learning. For example, the TFAD model (Zhang, Zhou, Wen and Sun, 2022) treats the time domain and frequency domain as two branches to jointly detect anomalies in time series. The TF-MAE model (Fang et al., 2024) designs a temporal-frequency dual-branch masked autoencoder network. The TF-C model (Zhang, Zhao, Tsiligkaridis and Zitnik, 2022) employs contrastive learning to align the time and frequency domains. Similarly, the Dual-TF model (Nam et al., 2024) designs a novel nested-sliding window to achieve time–frequency granularity alignment. Although these models can take advantage of the frequency domain, they must address the alignment problem of the time and frequency domains.

2.3. Spatio-temporal-based TSAD

A IIoT time series always contains multiple variables. There are the same or similar temporal patterns among multiple variables. To capture the coupling among multiple variables, some spatio-temporal-based TSAD models (He et al., 2023; He, Li, Wang, & Xiong, 2021) are proposed to enhance the accuracy of anomaly detection. Existing spatio-temporal-based TSAD models also can be divided into two categories.

One category uses a learning process to equally capture temporal dependency features and spatial coupling features from multivariable sensor signals. For example, the GST-Rro model (Zheng et al., 2024) designs a graph neural network to identify anomalies from the multivariable sensor signals with missing values. The FuGLAD model (He, Li, Xie and Sharma, 2024) combines multiple graph structures and prior knowledge to capture the coupling between variables. The MSTGAD model (He, Guo, Li, Xie, & Sharma, 2025) employs multiple spatio-temporal graph convolution networks to extract correlations among sensor nodes. The GSLAD model (He, Li, Yi et al., 2024) uses full graph parameterization and diffusion convolutional recurrent neural network (DCRNN) to extract temporal and spatial features faster and more accurate. However, these methods need to build graph structures from raw data, which takes some extra time. In addition, parallel computing of graph structures is difficult, which makes the speed of these models unsatisfactory.

The other category takes temporal and spatial as two complementary branches and employs two separate learning processes to learn their features. For instance, the STADN model (Tian, Zhuo, Liu, Chen, & Zhou, 2023) develops two learning processes to perform temporal and spatial feature extraction, where a GAT network is utilized to learn the spatial coupling and a LSTM network is used to capture temporal dependence. The MSTE model (Liu et al., 2024) proposes memory augmentation mechanisms for temporal and spatial learning branches to improve detection accuracy. The STEN model (Chen et al., 2024) designs a order prediction-based temporal normality learning process and a distance prediction-based spatial normality learning process. However, these models consume more storage and computational resources, and they also ignore the different weights of temporal and spatial features in anomaly scoring.

2.4. Summary

In Industry 4.0, TSAD models for the Industrial Internet of Things (IIoT) must meet new requirements: high accuracy, high speed, and low resource consumption. Driven by these new requirements, a ideal IIoT TSAD model should have a lightweight architecture with shallow structures and few parameters. Therefore, the simple MLP network is a good choice for such a lightweight architecture. However, MLP networks are usually used to compress models for TSAD in most existing research works. Furthermore, these MLP networks based models still remain complex structures and huge parameters. Therefore, developing a lightweight All-MLP TSAD model with shallow structures and few parameters is a feasible and promising research direction. Nevertheless, this lightweight All-MLP TSAD model may have limited representation capabilities. Time–frequency domain joint learning offers an effective solution to address this limitation. By extracting features from both the time and frequency domains, the representation ability of the lightweight architecture is significantly enhanced. Moreover, since learning in the time and frequency domains is independent and parallelizable, it does not compromise the timeliness of the lightweight model.

3. Proposed model

3.1. Problem statement

An IIoT time series usually comes from T observations of an industrial dynamic system captured by M heterogeneous sensors, denoted as $\mathbf{X} = \{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_T\} \in \mathbb{R}^{M\times T}$. This time series has two dimensions: the time dimension and the variable (spatial) dimension.

- Time Dimension: The series is represented as $\mathbf{X} = \{x_t \in \mathbb{R}^{M\times 1}\}_{t=1}^{T}$, where x_t represents the observations at the tth timestamp from the M sensors.
- Variable (Spatial) Dimension: The series is represented as $\mathbf{X} = \{x^m \in \mathbb{R}^{1\times T}\}_{m=1}^{M}$, where x^m represents the observations from T timestamps at the mth sensor.

Problem statement: The objective of this paper is to develop a lightweight IIoT TSAD model. A time series $\mathbf{X}$ is divided into an unlabeled training set $\mathbf{X}^1$ and a labeled test set $\mathbf{X}^2$. The unlabeled set $\mathbf{X}^1$ is used to train the proposed TSAD model. After training, the model assigns a score to each timestamp in the test set $\mathbf{X}^2$:

$$Score_t = \text{LTFAD}\left(\boldsymbol{x}_t\right) \tag{1}$$

where $\boldsymbol{x}_t \in \mathbb{R}^M$ represents the observations at the tth timestamp in $\mathbf{X}^2$. This score is then used to determine whether the tth timestamp is an anomaly:

$$\bar{y}_t = \begin{cases} 1, & \text{if } \ Score_t \geq \alpha \\ 0, & \text{otherwise} \end{cases} \tag{2}$$

where α represents a preset threshold ranging from 0 to 1. If $\bar{y}_t = 1$, the tth timestamp is an anomaly; otherwise, it is considered normal.

3.2. Model overview

In this paper, a lightweight All-MLP time–frequency anomaly detection model, namely LTFAD, is proposed for IIoT time series. To achieve high speed and low resource consumption, a shallow All-MLP lightweight architecture is developed instead of the traditional bulky neural networks-based solutions. To improve the accuracy of this architecture, a dual-branch reconstruction network is constructed from two perspectives: "local to global reconstruction" and "global to local reconstruction". To further enhance the accuracy, the reconstruction learning process of each branch is performed in both time and frequency domains. To the best of our knowledge, this is the first work to develop a time–frequency anomaly detection model based only on the shallow All-MLP architecture.

The proposed LTFAD model consists of four main components, as illustrated in Fig. 2.

- *Variable-independent sampling.* This component quickly generates local and global neighbors for each timestamp. To facilitate parallel processing for each variable, variable-independent sampling is employed.
- *Local to global time–frequency reconstruction learning.* This component employs a shared two-layer MLP network based on real numbers and another shared two-layer MLP network based on complex numbers to build a lightweight All-MLP architecture as the first learning branch. In this branch, local neighbors of each timestamp are used to reconstruct its global neighbors in both the time and frequency domains. This component is executed in parallel on M variables, and the two MLPs within it are shared by M variables.
- *Global to local time–frequency reconstruction learning.* Similar to the abovementioned learning branch, this component also uses two shared two-layer MLP networks to construct the second learning branch. In this branch, global neighbors of each timestamp are used to reconstruct its local neighbors in both the time and frequency domains. This component is also executed in parallel on M variables, and the two MLPs within it are shared by M variables.
- *Time–frequency reconstruction error-based scoring.* This component combines dual-branch time–frequency reconstruction errors to generate an anomaly score for each timestamp.

3.3. Variable-independent sampling

In this section, a simple and fast variable-independent sampling strategy is designed to generate local and global neighbors for each timestamp. The sampling process for each variable and timestamp is performed independently and in parallel.

Using the tth timestamp of the mth variable as an example, the sampling process is shown in Fig. 3. *Firstly*, a neighbor sequence of length L, centered at timestamp t, is extracted as local neighbors $LN_t^m = \{x_i^m\}_{i=t-L/2}^{t+L/2}$. *Secondly*, another neighbor sequence of length G, centered at timestamp t, is extracted, where $L <$ G. *Finally*, the remaining timestamps in the second neighbor sequence, excluding the local neighbors, are identified as the global neighbors $GN_t^m = \{x_i^m\}_{i=t-G/2}^{t-G/2} + \{x_i^m\}_{i=t+L/2}^{t+G/2}$, with a total length of $(G - L)$.

3.4. Local to global time–frequency reconstruction learning

This paper aims to build a shallow and lightweight All-MLP TSAD model. However, compared with deep and bulky networks, the shallow and lightweight architecture may have weaker representation capabilities and suffer from poor detection accuracy. To address this problem, we focus on improving accuracy in two ways.

One way is to optimize the design of the All-MLP architecture. Typically, the similarity between local and global neighbors is weaker for outliers than that of normal points, as illustrated in Fig. 1. This observation implies that the similarity between local and global neighbors can guide the design of our lightweight architecture. Constrained by the shallow All-MLP design, reconstruction learning is selected for similarity calculation. Technically, we employ one and only one two-layer MLP network to construct a lightweight reconstruction learning architecture. Based on this architecture, a dual-branch reconstruction network is designed to learn the similarities between local and global neighbors from two perspectives: "local to global reconstruction" and "global to local reconstruction".

The other way is to incorporate time–frequency domain learning to enhance representation capability. Generally, time and frequency domains are two complementary perspectives for analyzing time signals. Recent studies (Xu, Zeng et al., 2024; Yi et al., 2024) demonstrate that time series with complex features in the time domain are always

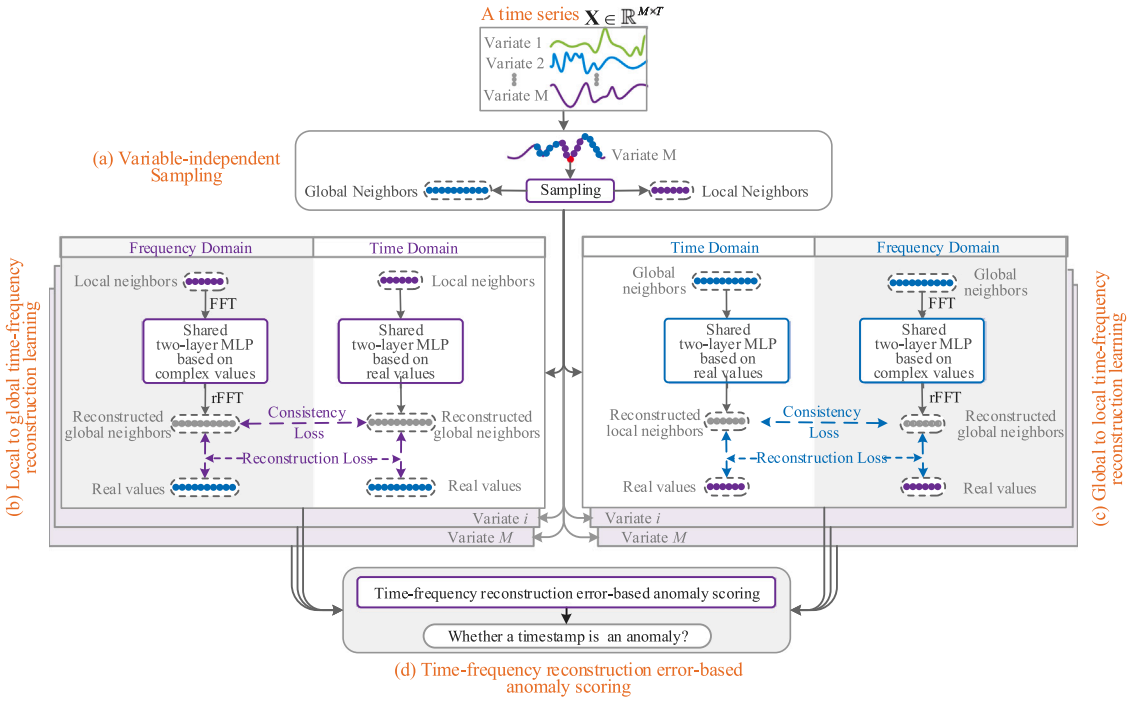


Fig. 2. Overview of the LTFAD model.

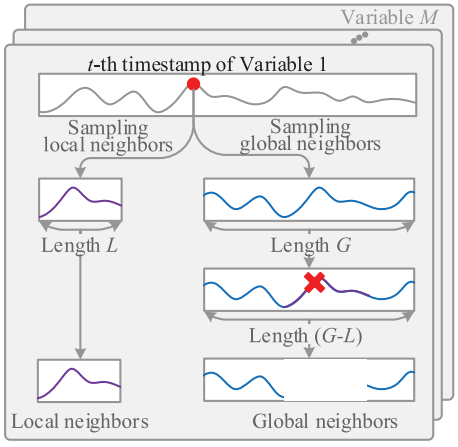


Fig. 3. Variable-independent sampling.

very simpler than that in the frequency domain. Therefore, we perform the reconstruction learning process of each branch in both the time and frequency domains, and align the two domains by a consistency loss. In this way, the representation capability of the dual-branch reconstruction network can be strengthened.

In this section, a local-to-global time–frequency reconstruction learning component is designed as the first branch of our dual-branch reconstruction network. In this branch, local neighbors of each timestamp are used to reconstruct its global neighbors in both time and frequency domains. The obtained reconstruction errors represent the similarity between two neighbors. Note that, the local-to-global reconstruction learning component is executed in parallel on M variables, and the two MLPs within it are shared by M variables.

3.4.1. Time-domain reconstruction

We employ only a two-layer MLP to perform local-to-global reconstruction learning in the time domain. This MLP includes an input layer with L neurons which is used to receive the local neighbors of each timestamp, a hidden layer with d neurons activated by *ReLU* and an output layer with $(G-L)$ neurons which is used to reconstruct the global neighbors. The input and hidden layers form the first fully connected network (FCN) with $L \times d$ trainable parameters. The hidden and output layers create the second FCN with $d \times (143G - L)$ trainable parameters. We take the tth timestamp of the mth variable as an example. The reconstruction learning process is described as follows.

Firstly, the local neighbors LN_t^m of length L are input into the first FCN to extract time-domain features, which is defined as follows.

$$\begin{aligned} LF_t^{Tim} &= \mathrm{ReLU}\left(LN_t^m \times \mathbf{W}_1^{Tim}\right) \\ \mathrm{ReLU}\,(x \in \mathbb{R}) &= \begin{cases} x, & if \quad x \geq 0 \\ 0, & otherwise \end{cases} \end{aligned} \tag{3}$$

where $LF_t^{Tim} \in \mathbb{R}^{1 \times d}$ is the output, and $\mathbf{W}_1^{Tim} \in \mathbb{R}^{L \times d}$ is the training parameters of the first FCN.

Secondly, the extracted time-domain features LF_t^{Tim} are passed into the second FCN to reconstruct the global neighbors of timestamp t.

$$\overline{GN}_t^m = LF_t^{Tim} \times \mathbf{W}_2^{Tim} \tag{4}$$

where $\overline{GN}_t^m \in \mathbb{R}^{1 \times (G-L)}$ denotes the reconstructed values, and $\mathbf{W}_2^{Tim} \in \mathbb{R}^{d \times (G-L)}$ refers to the parameters of the second FCN.

3.4.2. Frequency-domain reconstruction

To get frequency domain information, fast fourier transform (FFT) is employed as to convert time domain to frequency domain. Technically, a discrete time series with length of L in the time domain is transformed into L components in the frequency domain by the FFT function. However, the amplitudes of these L frequency components are symmetrical centered on the $L/2$ position. The frequency component at the $L/2$ position is the DC component. In this case, the first component is conjugate to the last component. Therefore, only the first $L/2$ frequency components are used as the frequency domain information. It not only reduces the learning of redundant information, but also reduces model parameters and improves timeliness.

Similar to time-domain reconstruction, another complex number-based two-layer MLP is adopted for frequency-domain reconstruction learning. This MLP consists of an input layer with $L/2$ neurons, a

hidden layer with d neurons activated by *zReLU*, and an output layer with $(G-L)/2$ neurons. In this network, the first FCN contains $(L/2)\times d$ parameters, and the second FCN contains $d\times[(G-L)/2]$ parameters. Unlike time-domain MLP, the frequency-domain MLP is based on complex numbers, where each training parameter is a complex number rather than a real number. Taking the tth timestamp of the mth variable as an example, the reconstruction process is presented as below.

Firstly, the Fast Fourier Transform(FFT) is employed to convert the local neighborhood sequence LN_t^m of length L into frequency components of length $L/2$.

$$FC_t^m = \mathrm{FFT}\left(LN_t^m\right) \tag{5}$$

where each frequency component is a complex number, $LN_t^m \in \mathbb{R}^{1\times L} \rightarrow FC_t^m \in \mathbb{C}^{1\times L/2}$.

Secondly, the frequency components FC_t^m are fed into the first FCN to generate frequency-domain features.

$$\begin{aligned} LF_t^{Fre} &= \mathrm{zReLU}\left(FC_t^m \times \mathbf{W}_1^{Fre}\right) \\ \mathrm{zReLU}\left(z\in\mathbb{C}\right) &= \begin{cases} z, & if \quad \theta_z \in [0,\pi/2] \geq 0 \\ 0, & otherwise \end{cases} \end{aligned} \tag{6}$$

where $LF_t^{Fre} \in \mathbb{C}^{1\times d}$ represents the output, and $\mathbf{W}_1^{Fre} \in \mathbb{C}^{(L/2)\times d}$ are the parameters of the first FCN. θ_z refers to the phase of the complex z.

Thirdly, the frequency-domain features LF_t^{Fre} are passed through the second FCN to reconstruct the frequency components of the global neighbors.

$$GF_t^m = LF_t^{Fre} \times \mathbf{W}_2^{Fre} \tag{7}$$

where $GF_t^m \in \mathbb{C}^{1\times[(G-L)/2]}$ is the output, and $\mathbf{W}_2^{Fre} \in \mathbb{C}^{d\times[(G-L)/2]}$ are the parameters of the second FCN.

Finally, the Inverse Fast Fourier Transform(rFFT) is applied to transform the reconstructed frequency components GF_t^m back into the global neighbors for timestamp t.

$$\overline{\overline{GN}}_t^m = \mathrm{rFFT}\left(GF_t^m\right) \tag{8}$$

where $\overline{\overline{GN}}_t^m \in \mathbb{R}^{1\times(G-L)}$ is the reconstructed values from the frequency domain.

Notably, the reconstruction process for each time stamp within a variable is independent of each other. Similarly, the reconstruction process of the M variables is also independent and can be calculated in parallel.

3.4.3. Loss

To train the above time-domain and frequency-domain MLPs, we construct a loss function consisting of three parts.

The first is a reconstruction loss of global neighbors in the time domain, which is defined as.

$$TL = \frac{1}{T}\sum_{t=1}^{T}\frac{1}{M}\sum_{m=1}^{M}\frac{1}{G-L}\sum_{i=1}^{G-L}\left(\overline{GN}_t^m(i) - GN_t^m(i)\right)^2 \tag{9}$$

where T is the number of timestamps, M is the number of variables, $(G-L)$ is the length of global neighbors, $\overline{GN}_t^m(i)$ and $GN_t^m(i)$ are the time-domain reconstructed and real values of the ith global neighbor at timestamp t for the mth variable, respectively.

The second is a reconstruction loss of global neighbors in the frequency domain, which is expressed as.

$$FL = \frac{1}{T}\sum_{t=1}^{T}\frac{1}{M}\sum_{m=1}^{M}\frac{1}{G-L}\sum_{i=1}^{G-L}\left(\overline{\overline{GN}}_t^m(i) - GN_t^m(i)\right)^2 \tag{10}$$

where $\overline{\overline{GN}}_t^m(i)$ is the frequency-domain reconstructed value.

The last one is a consistency loss to align time domain and frequency domain.

$$TFL = \frac{1}{T}\sum_{t=1}^{T}\frac{1}{M}\sum_{m=1}^{M}\frac{1}{G-L}\sum_{i=1}^{G-L}\left(\overline{\overline{GN}}_t^m(i) - \overline{GN}_t^m(i)\right)^2 \tag{11}$$

This loss aims to minimize the difference between the reconstructed values in the time and frequency domains.

3.5. Global to local time–frequency reconstruction learning

In this section, a global-to-local reconstruction learning component is designed as the second branch of the dual-branch reconstruction network. It measures the similarity between two neighbors from another perspective of "reconstructing local neighbors using global neighbors", which complements the first branch. Note that, the global-to-local reconstruction learning component is also executed in parallel on M variables, and the two MLPs within it are shared by M variables.

3.5.1. Time-domain reconstruction

Similar to the first branch, a real number-based two-layer MLP is employed. It consists of an input layer with $(G-L)$ neurons, a hidden layer with d neurons and an output layer with L neurons. In this MLP, the first FCN contains $(G-L)\times d$ parameters, and the second FCN contains $d\times L$ parameters. The reconstruction process at the tth timestamp for the mth variable is presented as follows.

Firstly, the global neighbors GN_t^m of length $(G-L)$ are fed into the first FCN to extract time-domain features.

$$GF_t^{Tim} = \mathrm{ReLU}\left(GN_t^m \times \mathbf{W}_3^{Tim}\right) \tag{12}$$

where $GF_t^{Tim} \in \mathbb{R}^{1\times d}$ is the output, and $\mathbf{W}_3^{Tim} \in \mathbb{R}^{(G-L)\times d}$ are the parameters of the first FCN.

Then, the GF_t^{Tim} are input into the second FCN to reconstruct the local neighbors of timestamp t.

$$\overline{LN}_t^m = GF_t^{Tim} \times \mathbf{W}_4^{Tim} \tag{13}$$

where $\overline{LN}_t^m \in \mathbb{R}^{1\times L}$ denotes the reconstructed values, and $\mathbf{W}_4^{Tim} \in \mathbb{R}^{d\times L}$ are the parameters of the second FCN.

3.5.2. Frequency-domain reconstruction

Another complex number-based two-layer MLP is employed. This MLP consists of an input layer with $(G-L)/2$ neurons, a hidden layer with d neurons, and an output layer with $L/2$ neurons. And, its first FCN contains $[(G-L)/2]\times d$ parameters, and its second FCN contains $d\times(L/2)$ parameters. The reconstruction process for the tth timestamp of the mth variable proceeds as follows.

Firstly, the FFT is employed to convert the global neighborhood sequence GN_t^m of length $(G-L)$ into frequency components of length $(G-L)/2$.

$$FC1_t^m = \mathrm{FFT}\left(GN_t^m\right) \tag{14}$$

where each frequency component is a complex number, $GN_t^m \in \mathbb{R}^{1\times(G-L)} \rightarrow FC1_t^m \in \mathbb{C}^{1\times[(G-L)/2]}$.

Secondly, the frequency components $FC1_t^m$ are input into the first FCN to get frequency-domain features.

$$GF_t^{Fre} = \mathrm{zReLU}\left(FC1_t^m \times \mathbf{W}_3^{Fre}\right) \tag{15}$$

where $GF_t^{Fre} \in \mathbb{C}^{1\times d}$ is the output, $\mathbf{W}_3^{Fre} \in \mathbb{C}^{[(G-L)/2]\times d}$ are the parameters of the first FCN.

Thirdly, the frequency-domain features GF_t^{Fre} are fed into the second FCN to reconstruct the frequency components of the local neighbors.

$$LF_t^m = GF_t^{Fre} \times \mathbf{W}_4^{Fre} \tag{16}$$

where $LF_t^m \in \mathbb{C}^{1\times L/2}$ represents the output, and $\mathbf{W}_4^{Fre} \in \mathbb{C}^{d\times L/2}$ are the parameters of the second FCN.

Finally, the rFFT is employed to generate the reconstructed local neighbors.

$$\overline{\overline{LN}}_t^m = \mathrm{rFFT}\left(LF_t^m\right) \tag{17}$$

Similar to the first branch, we also perform variable-independent learning for speed up.

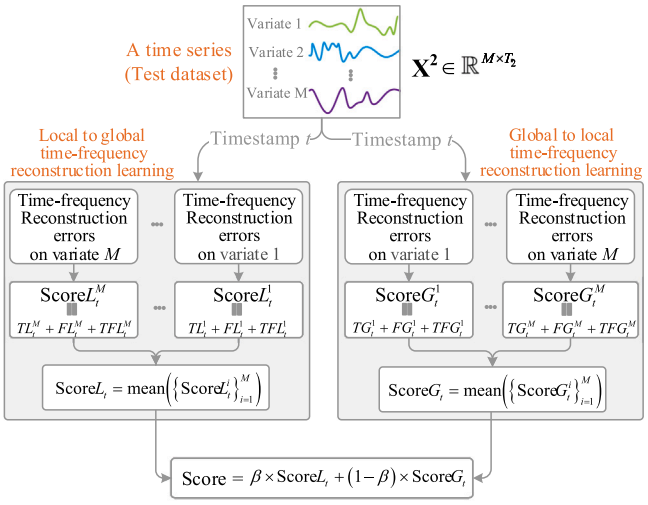


Fig. 4. Time–frequency reconstruction error-based anomaly scoring.

3.5.3. Loss

The loss function of this branch also consists of three parts. The time-domain reconstruction loss is defined as follows.

$$TG = \frac{1}{T}\sum_{t=1}^{T}\frac{1}{M}\sum_{m=1}^{M}\frac{1}{L}\sum_{i=1}^{L}\left(\overline{LN}_t^m(i) - LN_t^m(i)\right)^2 \tag{18}$$

where L is the length of local neighbors, $\overline{LN}_t^m(i)$ and $LN_t^m(i)$ are the time-domain reconstructed and real values.

The frequency-domain reconstruction loss is expressed as.

$$FG = \frac{1}{T}\sum_{t=1}^{T}\frac{1}{M}\sum_{m=1}^{M}\frac{1}{L}\sum_{i=1}^{L}\left(\overline{\overline{LN}}_t^m(i) - LN_t^m(i)\right)^2 \tag{19}$$

where $\overline{\overline{LN}}_t^m(i)$ is the frequency-domain reconstructed value.

The time–frequency consistency loss is defined as.

$$TFG = \frac{1}{T}\sum_{t=1}^{T}\frac{1}{M}\sum_{m=1}^{M}\frac{1}{L}\sum_{i=1}^{L}\left(\overline{\overline{LN}}_t^m(i) - \overline{LN}_t^m(i)\right)^2 \tag{20}$$

3.6. Time–frequency reconstruction error-based scoring

In this section, a customized time–frequency reconstruction error-based anomaly scoring strategy is designed to generate an anomaly score for each timestamp in the test set $\mathbf{X}^2$.

The detailed scoring process is shown in Fig. 4. *Firstly*, local neighbors and global neighbors of a timestamp t are fed into two reconstruction branches for parallel similarity calculation. *Secondly*, the similarity between two neighbors is computed across M variables in two branches. *Thirdly*, each branch produces three loss values per variable, and their sum is regarded as the similarity score on this variable. For example, in the first branch, $\text{Score}L_t^M = (TL_t^M + FL_t^M + TFL_t^M)$ represents the similarity score for the Mth variable at timestamp t, where TL_t^M, FL_t^M, and TFL_t^M denote time-domain, frequency-domain, and time–frequency consistency errors, respectively. *Fourthly*, average similarity scores $\text{Score}L_t = \text{mean}(\{\text{Score}L_t^i\}_{i=1}^M)$ and $\text{Score}G_t = \text{mean}(\{\text{Score}G_t^i\}_{i=1}^M)$ are calculated for two branches. *Finally*, these scores are merged with different weights to generate the final score at timestamp t.

$$\text{Score}_t = \beta \times \text{Score}L_t + (1-\beta) \times \text{Score}G_t \tag{21}$$

where $\beta \in [0,1]$ is a preset parameter. When $\beta = 0$, only the anomaly score of the second branch is considered. When $\beta = 1$, only the anomaly score of the first branch is considered.

After getting the anomaly score, Eq. (2) is employed to determine whether the tth timestamp is an anomaly.

3.7. Algorithm and analysis

The pseudo-code for LTFAD is shown in Algorithm 1.

Complexity: The computational costs of the LTFAD model involves three stages: variable-independent sampling, local-to-global time-frequency reconstruction learning, global-to-local time–frequency reconstruction learning. *In the first stage*, the global and local neighbors of each timestamp need to be generated. Moreover, the sampling process is performed in parallel on M variables. Therefore, the complexity of this stage is $\mathcal{O}(T \times G)$, where T is the number of timestamps, $G - L$ is the number of global neighbors of a timestamp, L is the number of local neighbors. *In the second stage*, a two-layer MLP based on real numbers and another two-layer MLP based on complex numbers are employed to perform local-to-global reconstruction learning for each timestamp from time domain and frequency domain, respectively. The local-to-global time–frequency reconstruction learning process is performed in parallel on M variables. Specifically, in time domain learning, the local neighbors of each timestamp are used to reconstruct its global neighbors. Thus, the complexity is $\mathcal{O}\left(T \times \left(\underbrace{L \times d}_{1\text{st}-layer} + \underbrace{(G-L) \times d}_{2nd-layer}\right)\right)$, where d is the number of neurons in the hidden layer, $L \times d$ is the complexity of 1st-layer in the MLP network, $(G-L) \times d$ is the complexity of 2nd-layer. Similarly, the complexity for the frequency domain learning is $\mathcal{O}\left(T \times \left(\underbrace{G \times \log_G}_{FFT+rFFT} + \underbrace{(L/2) \times d}_{1\text{st}-layer} + \underbrace{((G-L)/2) \times d}_{2nd-layer}\right)\right)$. Therefore, the total complexity of this stage is $\mathcal{O}\left(T \times \left(\frac{3}{2} \times G \times d + G \times \log_G\right)\right)$. *In the third*

Algorithm 1: LTFAD

input : Time-series data $\mathbf{X} \in \mathbb{R}^{M \times T}$, Training set $\mathbf{X}^1$, Test set $\mathbf{X}^2$, and $\mathbf{X} = \mathbf{X}^1 \cup \mathbf{X}^2$; parameter α.
output: Anomaly scores for all timestamps in $\mathbf{X}^2$.

```
// Time--frequency dual-branch learning
for t ∈ {1, ..., T} in X^1 do
    // Variable-independent learning in parallel
    for m ∈ {1, ..., M} parallel do
        // Sampling
        LN_t^m ← generate local neighbors for x_t^m;
        GN_t^m ← generate global neighbors for x_t^m;
        // Local to global learning
        // Two MLPs are shared by M variables
        GN̄_t^m ← 1st-MLP(LN_t^m);
        GN̿_t^m ← rFFT(2nd-Comp-MLP(FFT(LN_t^m)));
        Loss ← TL_t^m + FL_t^m + TFL_t^m by Eqs.(9) ~ (11);
        // Global to local learning
        // Two MLPs are shared by M variables
        LN̄_t^m ← 3rd-MLP(GN_t^m);
        LN̿_t^m ← rFFT(4th-Comp-MLP(FFT(GN_t^m)));
        Loss ← TG_t^m + FG_t^m + TFG_t^m by Eqs.(18) ~ (20);
    end
end
// Anomaly Scoring
for t ∈ {1, ..., T} in X^2 do
    ScoreL_t = mean({ScoreL_t^i = TL_t^i + FL_t^i + TFL_t^i}_{i=1}^M);
    ScoreG_t = mean({ScoreG_t^i = TG_t^i + FG_t^i + TFG_t^i}_{i=1}^M);
    Score_t = β × ScoreL_t + (1 − β) × ScoreG_t by Eqs.(21);
end
Return Score.;
```

Table 1
Datasets.

	Dataset	Features	Train set	Test set	URL
Multivariate	SKAB	8	12 450	5710	SKAB-URL
	Genesis	18	9732	16 220	Genesis-URL
	MSL	55	58 317	73 729	MSL-URL
	PSM	25	132 481	87 841	PSM-URL
	SWaT	51	495 000	449 919	SWaT-URL
	WADI	127	103 680	172 801	WaDi-URL
	PUMP	51	132 192	88 128	PUMP-URL
Unvariate	Dodgers	1	30 240	50 400	Dodgers-URL
	ECG	1	229 900	229 900	ECG-URL
	NAB	1	13 616	22 694	NAB-URL
	SensorScope	1	30 356	30 356	SensorScope-URL
	SVDB	1	184 319	46 080	SVDB-URL

stage, another two MLPs are employed to perform global-to-local reconstruction learning for each timestamp from time domain and frequency domain, respectively. The complexity of this stage is the same as the first stage, which is $\mathcal{O}\left(T \times \left(\frac{3}{2} \times G \times d + G \times \log_G\right)\right)$. *In summary*, the overall complexity of the LTFAD model is $\mathcal{O}\left(T \times \left(1 + 3 \times d + 2 \times \log_G\right) \times G\right)$. Ignoring relatively small factors$(1, 2, 3)$, the final complexity of our LTFAD model is $\mathcal{O}\left(T \times \left(d + \log_G\right) \times G\right)$, which is nearly linear because $\left(d + \log_G\right) \times G \ll T$.

4. Experiment

4.1. Experimental goal

We conduct extensive experiments to answer the following questions.

•*RQ-1 (Accuracy)*: Can our model achieve higher detection accuracy than baseline models on univariate and multivariate IIoT time-series datasets?

•*RQ-2 (Deployability)*: Can our model be easily deployed on resource-constrained edge IIoT devices?

•*RQ-3 (Timeliness and resource consumption)*: How does our model perform in terms of timeliness and resource consumption?

•*RQ-4 (Robustness)*: Is our model robust to noise?

•*RQ-5 (Ablation)*: Does each innovative component of our model benefit overall performance?

•*RQ-6 (Sensitivity)*: How do different parameter settings influence model performance?

4.2. Dataset

As listed in Table 1, twelve publicly available time-series are selected from the IIoT environments as our experimental datasets, including five univariate and seven multivariate datasets. These datasets cover various industrial domains, such as aviation, manufacturing, water treatment, and telemetry, for ensuring a fair and unbiased evaluation.

- **Multivariate datasets**. (1) SKAB: Multi-sensor temporal signals from an industrial testbed. (2) Genesis: Five continuous signals and thirteen discrete signals from IIoT multi-sensors. (3) MSL: Operating status of multiple sensors or controllers on the Mars rover. (4) PSM: 25-dimensional monitoring information from eBay Server Machines. (5) SWaT: 51-dimensional data collected by multiple sensors in public water treatment infrastructure. (6) WADI: 127-dimensional monitoring data from an industrial control system. (7) PUMP: Inspection data from multiple sensors on pumps.
- **Univariate datasets**. (1) Dodgers: Traffic flow data collected by induction loop sensors near Dodger Stadium, Los Angeles. (2) ECG: Electrocardiogram data with abnormal ventricular premature beats. (3) NAB: Operational status records of a server cluster. (4) SensorScope: Weather monitoring data from a wireless sensor network. (5) SVDB: 78 half-hour ECG tracings with supraventricular arrhythmia abnormalities.

4.3. Baseline, metric, and environment

4.3.1. Baseline

As listed in Table 2, ten representative state-of-the-art (SOTA) models are selected as competitors of our LTFAD model. (1) *Considering model structure:* DCdetector, ATF-UAD, TFMAE, PatchAD, DTAAD and SimAD are deep anomaly detection models with multi-layer structures and numerous parameters. In contrast, DIFFI, COUTA, LODA, STEN and LTFAD are shallow models. (2) *Regarding time and frequency domains:* DCdetector, DIFFI, COUTA, LODA, PatchAD, DTAAD, STEN and SimAD operate solely in the time domain. Conversely, ATF-UAD, TFMAE and LTFAD combine time and frequency domains to perform anomaly detection. (3) *Focusing on backbone:* The DIFFI, COUTA, and LODA are machine learning-based models, whereas the remaining models are neural network-based.

4.3.2. Metric

To comprehensively evaluate our model, four widely used metrics are employed: Accuracy (ACC), Precision, Recall, and F1-Score (F-Score).

•*ACC* measures the ratio of correctly predicted samples to the total sample count.

•*Precision* indicates the proportion of accurately predicted normal timestamps among all predicted normal timestamps.

•*Recall* reflects the proportion of actual normal timestamps that are correctly identified.

•*F-Score* provides a balanced evaluation by combining *Recall* and *Precision*. In general, these metrics range from 0 to 1, with higher values indicating better anomaly detection performance.

4.3.3. Experimental environment

The main experiments are conducted in a Windows 11 environment equipped with an Intel(R) Core(TM) i7-10700KF CPU, an NVIDIA GeForce RTX 3090 GPU (24 GB VGA-RAM), and 64 GB RAM. The deployment experiments are performed on two resource-limited IIoT edge devices (Raspberry Pi 4b and NVIDIA Jetson Xavier NX). The LTFAD model is developed in Python using PyCharm as the integrated development environment (IDE). The source code is publicly available at https://github.com/infogroup502/LTFAD. For the other models, official Python implementations are obtained from the websites of respective authors, as detailed in Table 2.

4.4. RQ-1 (Accuracy)

To answer *RQ-1*, the LTFAD model is compared with ten baselines across twelve public IIoT datasets, including five univariate and seven multivariate time-series.

4.4.1. Accuracy on univariate datasets

Table 3 lists the detection accuracy of ten models on five univariate datasets. In the table, bold text indicates the best performance, while underlined text denotes the second-best performance. From the table, several findings emerge.

- *Overall performance*: LTFAD, TFMAE, COUTA, DCdetector, STEN and ATF-UAD demonstrate better overall performance compared to other models. For example, on the SensorScope dataset, LTFAD achieves an ACC of 0.9921 and an F-Score of 0.9815, followed closely by DCdetector (0.9920 and 0.9813), while SimAD performs the poorest (0.3335 and 0.3347).

Table 2
Baselines.

Models	Full name	Year	Source code
COUTA (Xu, Wang et al., 2024)	Calibrated One-class Classification for Unsupervised Time Series Anomaly Detection	2024	github.com/xuhongzuo/couta
PatchAD (Zhong, Yu, Yang et al., 2024)	PatchAD: A Lightweight Patch-Based MLP-Mixer for Time Series Anomaly Detection	2024	github.com/EmorZz1G/PatchAD
SimAD (Zhong, Yu, Xi et al., 2024)	SimAD: A Simple Dissimilarity-based Approach for Time Series Anomaly Detection	2024	github.com/EmorZz1G/SimAD
TFMAE (Fang et al., 2024)	Temporal-Frequency Masked Autoencoders for Time Series Anomaly Detection	2024	github.com/LMissher/TFMAE
DTAAD (rui Yu, hong Lu, & Xue, 2024)	DTAAD: Dual Tcn-Attention Networks for Anomaly Detection in Multivariate Time Series Data	2024	github.com/Yu-Lingrui/DTAAD
ATF-UAD (Fan, Wang et al., 2023)	An Adversarial Time–Frequency Reconstruction Network for Unsupervised Anomaly Detection	2023	github.com/wzhSteve/ATF-UAD
DCdetector (Yang et al., 2023)	DCdetector: Dual Attention Contrastive Representation Learning for Time Series Anomaly Detection	2023	github.com/DAMO-DI-ML/KDD2023-DCdetector
DIFFI (Carletti et al., 2023)	Interpretable anomaly detection with diffi: Depth-based feature importance of isolation forest	2023	github.com/mattiacarletti/DIFFI
LODA (Pevnỳ, 2016)	Loda: Lightweight on-line detector of anomalies	2016	github.com/OpsPAI/MTAD
STEN (Chen et al., 2024)	Self-Supervised Spatial-Temporal Normality Learning for Time Series Anomaly Detection	2024	github.com/mala-lab/STEN

Table 3
Performance analysis on univariate datasets.

		LTFAD	COUTA	DIFFI	PatchAD	SimAD	TFMAE	DCdetector	DTAAD	LODA	ATF-UAD	STEN
Dodgers	ACC	**0.9648**	0.9366	0.3444	0.9253	0.9085	0.9620	0.9609	0.8931	0.8654	0.8974	0.9645
	Precision	0.8896	**0.9249**	0.1138	0.8211	0.4273	0.8500	0.8732	0.4571	0.4551	0.4881	0.8198
	Recall	0.7328	0.3859	0.8359	0.3061	0.2539	0.7421	0.6742	0.4732	0.3553	**0.9152**	0.8182
	F-Score	**0.8037**	0.5445	0.2003	0.4459	0.3185	0.7924	0.7721	0.4650	0.3990	0.6367	0.8019
ECG	ACC	**0.9851**	0.8300	0.8201	0.9609	0.2439	0.9721	0.9650	0.9647	0.9815	0.9673	0.9587
	Precision	**0.9496**	0.4596	0.4308	0.8589	0.1646	0.9212	0.9332	0.8213	0.8973	0.8324	0.8900
	Recall	0.9568	0.4068	0.4163	0.9014	0.9211	0.9013	0.8396	**0.9938**	0.9544	**0.9938**	0.8439
	F-Score	**0.9532**	0.4316	0.4234	0.8797	0.2793	0.9112	0.8839	0.8993	0.9388	0.9060	0.8663
NAB	ACC	**0.9961**	0.9171	0.8096	0.9368	0.9288	0.9958	0.9956	0.2076	0.9813	0.9872	0.9827
	Precision	0.9700	0.6254	0.3943	0.9647	0.6763	0.9675	0.9659	0.1360	**1**	0.9070	0.8782
	Recall	**1**	0.8348	0.9814	0.5000	0.9302	**1**	**1**	**1**	0.8694	**1**	**1**
	F-Score	**0.9848**	0.7151	0.5625	0.6655	0.7832	0.9835	0.9826	0.2394	0.9302	0.9513	0.9352
SensorScope	ACC	**0.9921**	0.9877	0.7094	0.8298	0.3335	0.9917	0.9920	0.5860	0.5624	0.8902	0.9848
	Precision	**0.9846**	0.9460	0.3263	0.9112	0.2349	0.9629	0.9843	0.3313	0.8935	0.7424	0.9339
	Recall	0.9784	**1**	0.3322	0.2329	0.9250	**1**	0.9784	0.9110	0.3163	0.7484	**1**
	F-Score	**0.9815**	0.9723	0.3293	0.3710	0.3747	0.9811	0.9813	0.4858	0.4672	0.7454	0.9658
SVDB	ACC	**0.9950**	0.9940	0.9352	0.9869	0.9766	0.9905	0.9876	0.9937	0.9949	0.9923	0.9804
	Precision	0.7827	0.8166	0.6221	**0.9750**	0.2343	0.6551	0.6125	0.8047	0.7200	0.7508	0.4801
	Recall	**1**	0.8618	0.2839	0.2812	0.1298	**1**	0.8570	0.8618	**1**	0.8618	**1**
	F-Score	**0.8781**	0.8386	0.7793	0.4366	0.1671	0.7916	0.7144	0.8323	0.8372	0.8025	0.6487

- *Stability*: LTFAD exhibits the most stable performance across four metrics, with TFMAE, DCdetector, STEN, and ATF-UAD following. For instance, on the ECG, SensorScope and NAB datasets, LTFAD yields three first-place rankings and one second-place ranking.
- *Time–frequency joint learning*: Time–frequency joint learning-based models (LTFAD, TFMAE, and ATF-UAD) offer higher detection accuracy than other models based only on the time domain. Among these three models, LTFAD ranks highest.
- *Lightweight structures*: Deep models (PatchAD, TFMAE, DCdetector, DTAAD, ATF-UAD, SimAD) generally outperform shallow lightweight models(LODA, DIFFI, COUTA). However, our lightweight LTFAD exceeds multiple deep models. For example, on the NAB dataset, Precision scores are 0.9700 for LTFAD, 0.9659 for DCdetector, 0.9675 for TFMAE, 0.6254 for COUTA, and only 0.3943 for DIFFI.

4.4.2. Accuracy on multivariate datasets

Table 4 presents the detection accuracy of ten models on seven multivariate time-series. From the table, we can get similar observations.

- *Overall performance*: LTFAD consistently demonstrates superior performance, followed by TFMAE, DCdetector, PatchAD, STEN and ATF-UAD. For example, on the SKAB dataset, LTFAD achieves an ACC of 0.9974 and a Recall of 1, while COUTA only scores an ACC of 0.9121 and a Recall of 0.7775.
- *Stability*: LTFAD, TFMAE, DCdetector, STEN and PatchAD exhibit relatively stable accuracy across four metrics. For example, on the PSM dataset, LTFAD ranks first in three metrics and second in one, while DIFFI and SimAD display more variability.
- *Time–frequency joint learning*: TFMAE, LTFAD, and ATF-UAD, employing time–frequency learning, outperform time domain-only models like DIFFI, PatchAD, and DTAAD. For example, on the WADI dataset, the ACC is 0.9934 for both LTFAD and TFMAE, 0.9533 for DTAAD, and 0.6543 for DIFFI.
- *Lightweight structures*: The lightweight LTFAD model exceeds several deep and shallow models. For instance, on the PUMP dataset, LTFAD achieves a Precision of 0.9407 and an F-score of 0.9648, outperforming deep models such as PatchAD (Precision 0.9311, F-Score 0.9579) and DCdetector (Precision 0.9407, F-Score 0.9606).

Table 4
Performance analysis on multivariate datasets.

		LTFAD	COUTA	DIFFI	PatchAD	SimAD	TFMAE	DCdetector	DTAAD	LODA	ATF-UAD	STEN
SKAB	ACC	**0.9974**	0.9121	0.8184	0.9287	0.7896	0.9939	0.9938	0.9779	0.8469	0.9779	0.9876
	Precision	0.9926	0.9646	0.7443	**0.9969**	0.7372	0.9828	0.9828	0.9407	0.8005	0.9407	0.9657
	Recall	**1**	0.7775	0.7335	0.8005	0.627	**1**	**1**	**1**	0.7712	**1**	**1**
	F-Score	**0.9963**	0.8610	0.7389	0.8880	0.6777	0.9913	0.9913	0.9695	0.7856	0.9695	0.9826
Genesis	ACC	**0.9965**	0.9954	0.7666	0.9933	0.9645	0.9958	0.9952	0.9945	0.9957	**0.9965**	0.9801
	Precision	**0.6957**	0.6250	0.5265	0.5946	0.1138	0.6575	0.6234	0.5854	0.4400	0.6857	0.2743
	Recall	0.9600	**1**	0.8200	0.4400	**1**	0.9531	0.9522	0.9567	**1**	0.9425	0.9600
	F-Score	**0.8067**	0.7692	0.5146	0.5057	0.2044	0.7805	0.7559	0.7273	0.6111	0.8000	0.4267
MSL	ACC	**0.9901**	0.9763	0.8314	0.9653	0.4027	0.9894	0.9883	0.9244	0.9694	0.9432	0.9807
	Precision	0.9233	0.8988	0.6346	0.9479	0.1273	0.9192	0.9228	0.8007	**0.9531**	0.6531	0.8545
	Recall	**0.9885**	0.8737	0.6128	0.7098	0.7975	0.9860	0.9703	0.3756	0.7966	0.9830	0.9845
	F-Score	**0.9548**	0.8861	0.6235	0.8117	0.2195	0.9514	0.9460	0.5113	0.8679	0.7848	0.9149
PSM	ACC	**0.9911**	0.9133	0.7369	0.9399	0.5634	0.9902	0.9885	0.9270	0.9586	0.9021	0.9832
	Precision	0.9802	0.9445	0.5305	0.8843	0.3547	0.9788	0.9761	0.8415	0.8956	**0.9994**	0.9507
	Recall	**0.9880**	0.7306	0.4540	0.9012	0.8115	0.9859	0.9827	0.9080	0.9524	0.6476	0.9806
	F-Score	**0.9841**	0.8239	0.4893	0.8927	0.4937	0.9823	0.9794	0.8735	0.9231	0.7860	0.9703
SWaT	ACC	**0.9944**	0.9630	0.6331	0.9658	0.9509	0.9920	0.9913	0.7714	0.9622	0.9626	0.9755
	Precision	0.9557	0.9263	0.2233	**0.9842**	0.9543	0.9784	0.9331	0.3235	0.8035	0.9785	0.8580
	Recall	**1**	0.7555	0.8159	0.7303	0.6269	0.9556	**1**	0.8087	0.8752	0.7075	0.9568
	F-Score	**0.9774**	0.8323	0.3506	0.8384	0.7567	0.9669	0.9654	0.4621	0.8378	0.8212	0.9047
WADI	ACC	**0.9934**	0.9864	0.6543	0.9795	0.9538	**0.9934**	0.9915	0.9533	0.9729	0.9739	0.9810
	Precision	**0.8806**	0.8094	0.6152	0.8302	0.5806	0.8802	0.8510	0.5356	0.7241	0.6493	0.7176
	Recall	**1**	0.9393	0.9207	0.7241	0.2349	**1**	**1**	0.2496	0.7181	**1**	**1**
	F-Score	**0.9365**	0.8695	0.4048	0.7735	0.3345	0.9363	0.9195	0.3405	0.7211	0.7874	0.8356
PUMP	ACC	**0.9964**	0.9838	0.9766	0.9959	0.9836	0.9951	0.9961	0.9168	0.9956	0.9815	0.9812
	Precision	0.9407	0.7574	0.6843	0.9311	0.7754	0.9105	0.9407	0.8081	**0.9901**	0.7308	0.7247
	Recall	0.9905	0.9904	0.9784	0.9901	0.9403	**0.9998**	0.9729	0.864	0.9254	0.9901	**0.9998**
	F-Score	**0.9648**	0.8584	0.8053	0.9597	0.8499	0.9531	0.9606	0.8938	0.9566	0.8409	0.8403

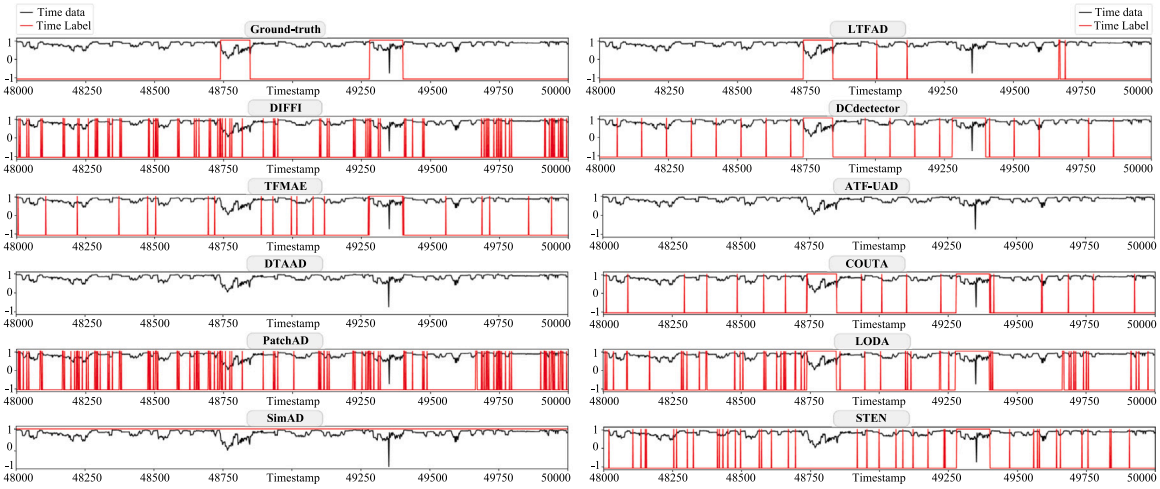


Fig. 5. Visualization of anomaly detection at timestamps 48 000–50 000 of 1th-variable on *MSL* dataset.

4.4.3. Visualization

Fig. 5 plots the detection results of all models for timestamps 48 000 to 50 000 of the first variable in the MSL dataset. In the figure, the black line represents the real data curve, while the red line indicates the corresponding label(peaks for anomalies, valleys for normal states). From the figure, we can obtain two findings.

- *Accuracy:* COUTA, DCdetector, LTFAD, TFMAE, and LODA align more closely with the real labels compared to other models. For example, anomalies in the range of 48 700 to 48 950 are accurately identified by these models, whereas other models produce numerous errors.
- *False alarms:* LTFAD, DCdetector, and LODA provide fewer false alarms than other models, with LTFAD performing particularly well.

4.4.4. In summary

The experimental results demonstrate the excellent and stable accuracy of LTFAD on identifying anomalies in industrial IoT environments. Specifically, the proposed lightweight LTFAD model consistently ranks first or second across four metrics on multiple datasets, outperforming several deep models. These findings highlight the effectiveness of time–frequency joint learning for anomaly detection. These advantages are attributed to the fact that LTFAD employs two accuracy optimization strategies: "global to local" and "local to global" dual-branch reconstruction and time–frequency joint learning.

4.5. RQ-2 (deployability)

To address *RQ2*, we conduct deployment experiments for ten models on two IIoT edge devices: Raspberry Pi 4b (little resources) and Jetson Xavier NX (general resources). Specifically, the Raspberry Pi 4b has a

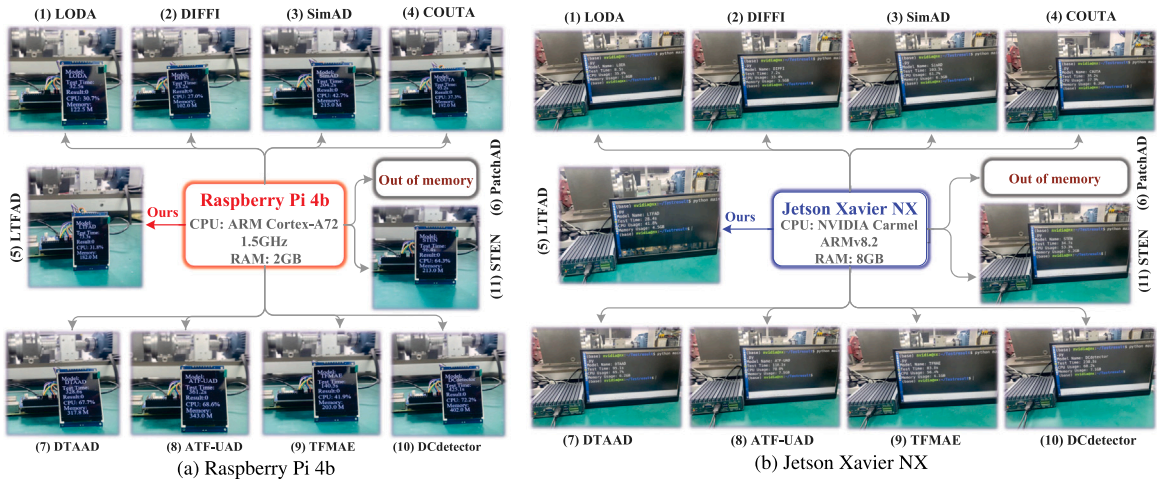


Fig. 6. Deployment results on two IIoT edge devices.

Table 5
Timeliness and resource consumption.

		Shallow models					Deep models					
		LTFAD	LODA	DIFFI	COUTA	STEN	PatchAD	SimAD	TFMAE	DCdetector	ATF-UAD	DTAAD
PC with powerful resources (CPU: i7-10700KF GPU: RTX 3090 RAM: 64G VGA-RAM: 24G)	Parameters	75.0 K	–	–	106.1 K	793.9 K	1230.7 K	1418.1 K	706.0 K	236.2 K	1726.2 K	123.4 K
	One-epoch-time	37.4 s	10.5 s	8.8 s	65.1 s	69.9 s	461.2 s	107.1 s	148.9 s	300.1 s	170.1 s	156.6 s
	CPU Usage	10.8%	6.7%	3.4%	37.3%	68.4%	6.6%	11.7%	9.9%	9.2%	7.6%	7.7%
	GPU Usage	15.0%	0%	3%	17.5%	76.3%	83.3%	41.2%	53.6%	58.0%	91.9%	83.8%
	RAM-Usage	7.1 GB	7.4 GB	6.5 GB	10.6 GB	9.4 GB	16.3 GB	10.8 GB	10.1 GB	12.7 GB	14.5 GB	14.0 GB
	VGA-RAM-Usage	2.2 GB	0.7 GB	0.9 GB	1.2 GB	3.9 GB	12.8 GB	4.8 GB	5.3 GB	10.7 GB	9.6 GB	7.1 GB
Jetson Xavier NX with general resources (CPU: Carmel ARMv8.2, RAM: 8G)	10000-timestamps-time	28.4 s	8.5 s	7.1 s	35.2 s	34.7 s	Ot	103.2 s	83.9 s	230.0 s	110.1 s	95.6 s
	CPU-Usage	41.8%	35.0%	33.4%	37.3%	53.3%	Ot	61.7%	56.4%	68.2%	78.0%	65.7%
	RAM-Usage	4.5 GB	3.8 GB	3.4 GB	6.2 GB	5.2 GB	Ot	6.3 GB	6.1 GB	7.1 GB	7.5 GB	6.2 GB
Raspberry Pi 4b with little resources (CPU: ARM Cortex-A72, RAM: 2G)	100-timestamps-time	73.3 s	32.5 s	23.1 s	93.2 s	96.4 s	Ot	204.2 s	140.5 s	425.1 s	361.2 s	328.6 s
	CPU-Usage	31.8%	30.7%	27%	37.3%	64.3%	Ot	42.7%	41.9%	72.2%	68.6%	67.7%
	RAM-Usage	182 MB	123 MB	102 MB	192 MB	213M	Ot	215 MB	203 MB	402 MB	343 MB	318 MB

1.5 GHz ARM Cortex-A72 processor and *2 GB* RAM, while the Jetson Xavier NX is equipped with a 6-core Carmel ARMv8.2 processor and 8 GB RAM. In this experiment, all models are initially trained in the cloud and subsequently deployed for testing on the edge devices. The deployment results are illustrated in Fig. 6. From the figure, two observations can be found.

- *Deployability:* ten models are successfully deployed, while PatchAD fails due to its storage requirements exceeding the limits of both edge devices. In comparison, lightweight models are easier to deploy.
- *Convenience:* In IIoT environments, frequent changes in production processes may result in dynamic fluctuations in time signals. Thus, we need to update the model regularly and keep its best performance. The update process involves fine-tuning the model with new data samples in the cloud and redeploying it to the edge device. During this process, lightweight models consume fewer data samples, less communication overhead and labor cost compared to deep models.
- *In summary:* Compared to deep models, shallow lightweight models are easier to deploy on resource-limited IIoT edge devices.

4.6. RQ-3 (Timeliness and resource consumption)

To answer *RQ-3*, we evaluate the timeliness and resource consumption of ten models on three deployment environments. These environments include a PC with powerful resources, Raspberry Pi 4b with little resources, and Jetson Xavier NX with general resources. In this experiment, model parameters, time for one epoch with same the *batch_size* (One-epoch-time), CPU and GPU usage (CPU-Usage and GPU-Usage), RAM utilization (RAM-Usage), and VGA RAM utilization (VGA-RAM-Usage) are chosen as evaluation metrics for the PC environment; The detection time for 100 timestamps (100-timestamps-time), CPU-Usage, and RAM-Usage are employed for the Raspberry Pi 4b environment; The processing time for 10 000 timestamps (10 000-timestamps-time), CPU-Usage, and RAM-Usage are employed for the Jetson Xavier NX environment. Table 5 lists the experimental results of the ten models, where "-" indicates no data available, and "Ot" represents out of memory. From the table, several important findings can be captured.

- *Training Parameters:* LTFAD has the fewest training parameters (75 K), compared to PatchAD (1230.7 K) and DTAAD (123.4 K).
- *Timeliness:* Shallow Lightweight models(LTFAD, LODA, DIFFI, STEN and COUTA) exhibit significantly faster processing times than deep models(PatchAD, SimAD, TFMAE, DCdetector, ATF-UAD, DTAAD) across all environments. For example, in the PC environment, the processing time is 37.4 s for LTFAD, only 8.8 s for DIFFI, but 461.2 s for PatchAD, 300.1 s for DCdetector.
- *Computing consumption:* Shallow models show substantially lower CPU and GPU usage compared to deep models. For example, the CPU-Usage on the Raspberry Pi 4b is 27% for shallow DIFFI, while 72.2% for deep DCdetector and 68.6% for deep ATF-UAD. Comparing the four shallow models, they have similar CPU usage.
- *Storage consumption*: Deep models take up significantly more storage resources. For example, on the Jetson Xavier NX environment with only 8 G RAM, the RAM-Usage is 7.5 GB for deep ATF-UAD, 7.1 GB for deep DCdetector, while 4.5 GB for our LTFAD and 3.4 GB for shallow DIFFI.

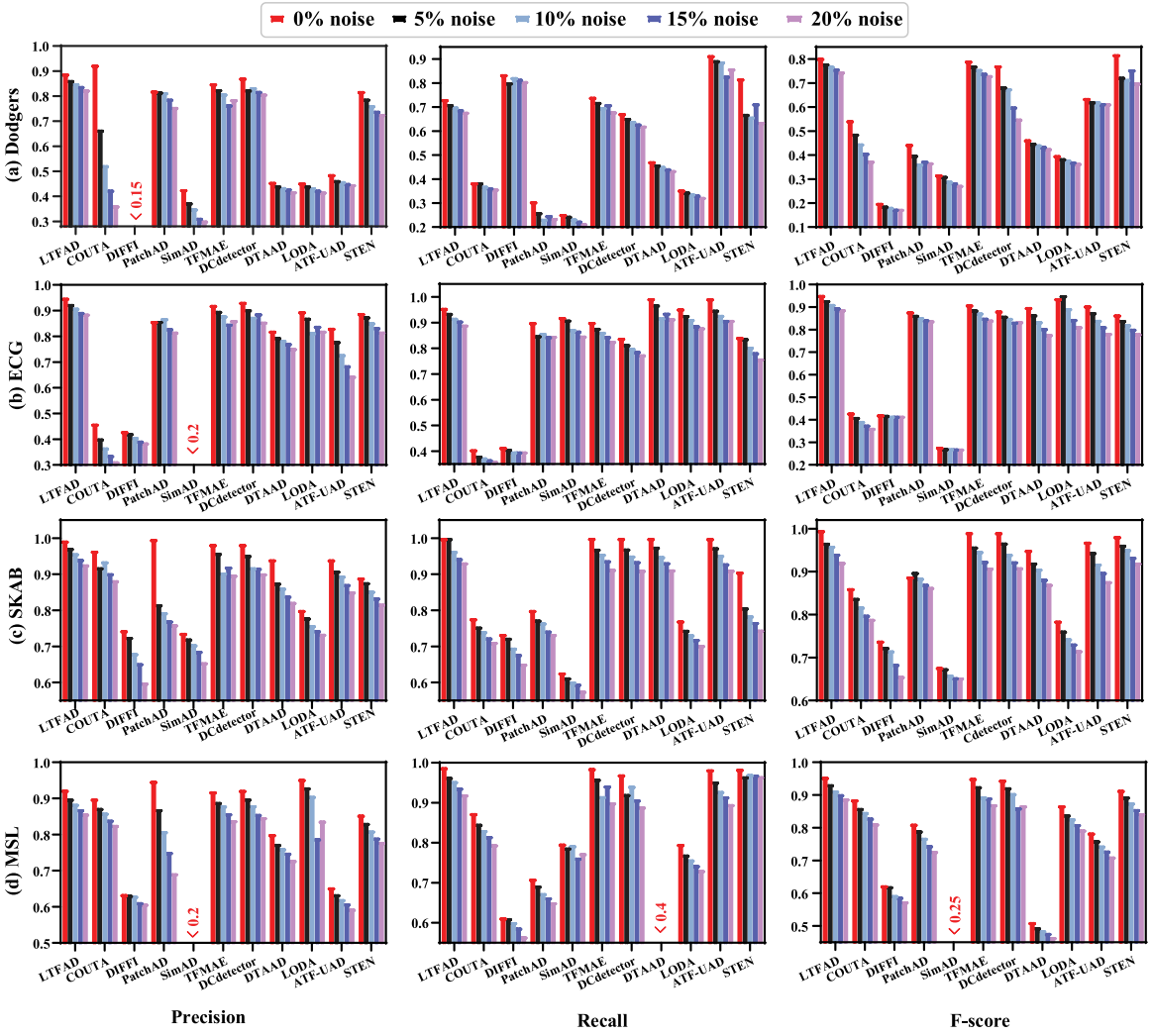


Fig. 7. Robustness analysis on noisy datasets.

- *In summary*: With the aid of shallow All-MLP architecture, LTFAD demonstrates good timeliness and low resource consumption, which makes it suitable for resource-limited IIoT edge devices.

4.7. RQ-4 (Robustness)

To address *RQ-4*, inspired by the latest assessment benchmarks (Paparrizos et al., 2022), we introduced different levels of artificial noise into the anomaly score calculation process to simulate a noisy environment. Specifically, we selected four datasets (Dodgers, ECG, SKAB, and MSL) to conduct robustness experiments with noise levels of 0%, 5%, 10%, 15%, and 20%.

Fig. 7 illustrates the performance changes of ten models on five noise environments. From the figure, several observations emerge.

- *Stability:* As noise levels increase, all models exhibit declining performance. In contrast, LTFAD, DCdetector, and TFMAE maintain more stable accuracy across the four datasets and five noise levels. For example, on the Recall metric, LTFAD and TFMAE exhibit significantly smaller performance fluctuations than other models.
- *Overall performance:* LTFAD, TFMAE, DCdetector, ATF-UAD, STEN, and DTAAD outperform compared to other models. For instance, on the SKAB dataset at 5% noise, the F-Score is 0.967 for LTFAD, 0.958 for TFMAE, 0.967 for DCdetector, 0.921 for DTAAD, 0.945 for ATF-UAD, with other models scoring lower.
- *In summary:* LTFAD demonstrates superior stability and performance across different noise environments. This indicates that LTFAD has strong noise tolerance.

4.8. RQ-5 (Ablation)

To answer *RQ-5*, we conduct three ablation experiments to evaluate the effectiveness of key components in LTFAD. In this experiment, we employ five IIoT time-series and three evaluation metrics (Precision, Recall and F-Score).

4.8.1. Dual-branch reconstruction learning

The similarity between local neighbors and global neighbors for outliers is typically weaker than for normal points. To efficiently capture these similarities, a novel two-branch reconstruction network with lightweight All-MLP architecture is constructed from two perspectives: “local to global reconstruction” and “global to local reconstruction”. Therefore, this experiment is designed to verify whether both branches contribute to the overall performance. In this experiment, we construct three competing schemes: only local to global reconstruction learning (1st scheme), only global to local reconstruction learning (2nd scheme), and dual-branch reconstruction learning (3rd scheme).

Fig. 8 plots the detection accuracy (Precision, Recall, and F-Score) of three schemes across five datasets. From the figure, several observations can be drawn.

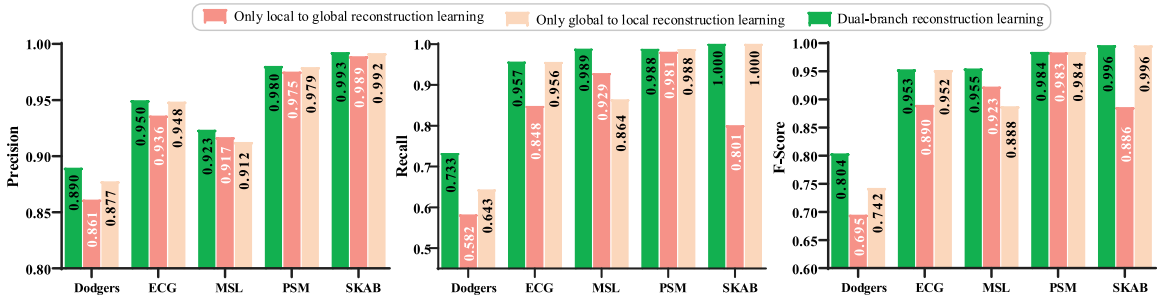


Fig. 8. Dual-branch reconstruction learning.

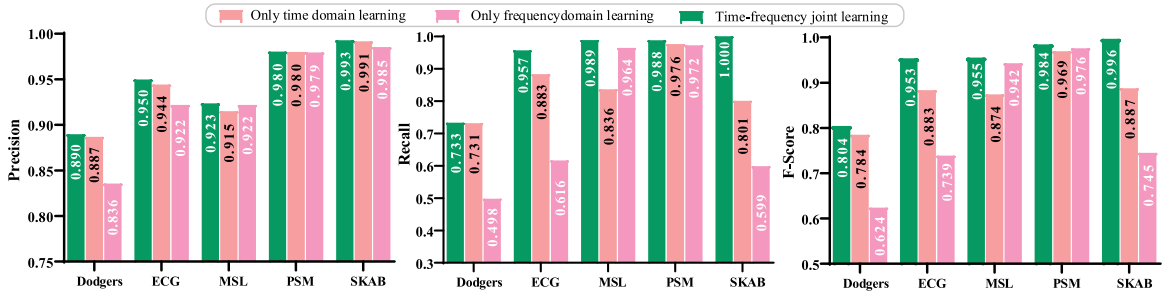


Fig. 9. Time–frequency joint learning.

- *Overall performance:* The dual-branch reconstruction learning (3rd scheme) outperforms the other schemes, having the highest bars across all metrics and datasets. For example, on the Dodgers and ECG datasets, the precision scores are 0.89 and 0.95 for the 3rd scheme, compared to 0.861 and 0.936 for the 2nd scheme, and 0.877 and 0.948 for the 1st scheme.
- *Comparing the 1st and 2nd schemes:* The global to local reconstruction learning (2nd scheme) demonstrates superior performance over the 1st scheme. On the SKAB dataset, the F-Score for the 2nd scheme is 0.996, significantly higher than the 0.886 of the 1st scheme.
- *In summary:* combining both branches (3rd scheme) yields the best overall performance.

4.8.2. Time–frequency joint learning

The time–frequency joint learning is another key innovation of LTFAD. Its purpose is to address the limitations of the lightweight All-MLP architecture in feature extraction and improve the model's accuracy. To evaluate its effectiveness, we design three contrasting models: 0.only time domain learning (1st model), only frequency domain learning (2nd model), and time–frequency joint learning (3rd model).

Fig. 9 displays the accuracy of the three models. From the figure, two key findings can be obtained.

- *Overall performance:* The time–frequency joint learning(3rd model) exhibits better performance across all three metrics and five datasets. For example, on the MSL dataset, the Recall score is 0.989 for the 3rd model, compared to 0.836 for the 1st model and 0.964 for the 2nd model. Similarly, on the PSM dataset, the 3rd model achieves an F-Score of 0.984, outperforming both the 1st model (0.969) and the 2nd model (0.976).
- *Comparison of the 1st and 2nd models:* The only time domain learning(1st model) is more efficient than the only frequency domain learning(2nd model). For instance, on the ECG dataset, the 1st model achieves higher scores in Precision, Recall, and F-Score.
- *In summary:* Both time domain and frequency domain learning contribute positively to model performance, and their combination yields the best results.

4.8.3. Frequency transformation

In the LTFAD model, fast fourier transform (FFT) is employed to convert time domain to frequency domain. Technically, a discrete time series with a length of L in the time domain is transformed into L components in the frequency domain by the FFT function. However, the amplitudes of these L frequency components are symmetrical centered on the $L/2$ position. The frequency component at the $L/2$ position is the DC component. In this case, the first component is conjugate to the last component. Therefore, the FFT function always only outputs the first $L/2$ frequency components as the frequency domain information of a time series. This experiment aims to verify the influence of the lost $L/2$ frequency components on the performance of the LTFAD model. In this experiment, two competing models are built: (1) LTFAD using only L/2 frequency components and (2) LTFAD using only L frequency components. Fig. 10 plots the accuracy of the two models across five datasets. From the figure, two observations can be concluded as below.

- *Overall performance:* The two models show very similar performance. For instance, on the Dodgers dataset, the first competing model achieves a Precision score of 0.8896 and a Recall score of 0.7328, while the second model has a Precision score of 0.8842 and a Recall score of 0.7328. Similarly, on the PSM dataset, the Recall score is 0.9880 for the first model, while 0.9863 for the second model.
- *Comparison of the 1st and 2nd models:* Each of them has advantages and disadvantages. For example, on the MSL dataset, the 1st model achieves higher Recall and F-Score scores, while the 2nd model gets the higher Precision score.
- *In summary:* The first model performs more stably and better. The main reason is that the conjugate L frequency components contain some redundant information, which reduces the accuracy of LTFAD.

4.9. RQ-6 (Sensitivity)

To address *RQ-6*, we evaluate the impact of three key parameters.

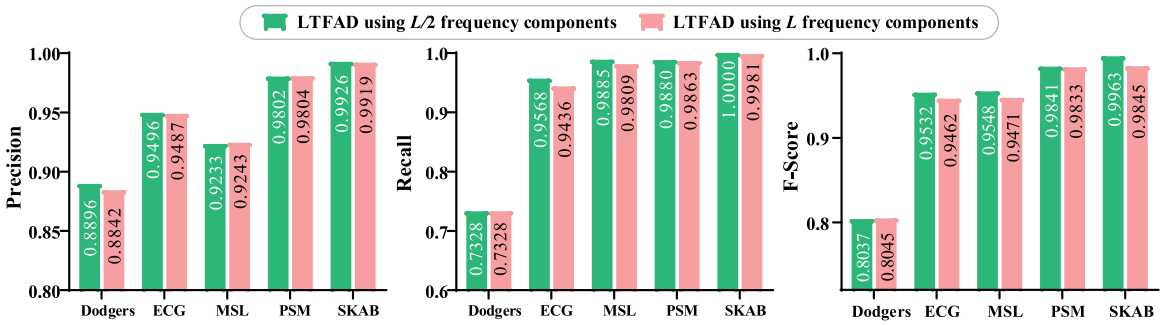


Fig. 10. Frequency transformation.

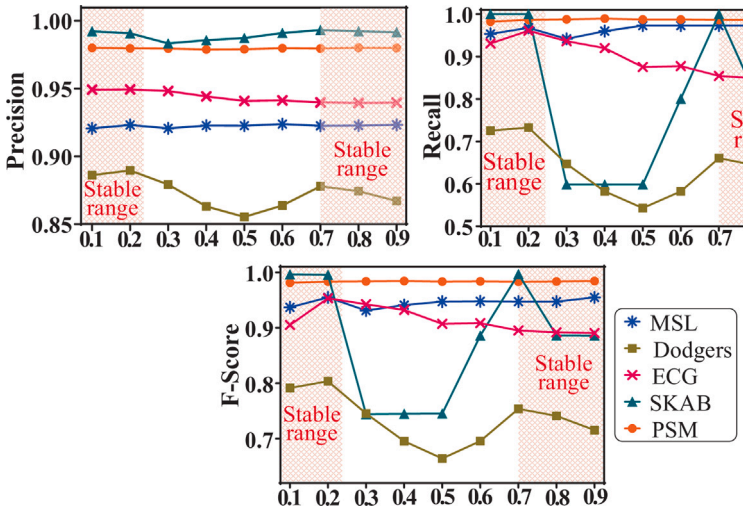


Fig. 11. Sensitivity for parameter β.

4.9.1. Parameter β

The first key parameter β, ranging from 0 to 1, is employed to merge the anomaly scores from two branches and generate the final anomaly score. When $\beta = 1$, only the anomaly score from the local-to-global reconstruction branch is used. When $\beta = 0$, only the anomaly score from the global-to-local reconstruction branch is used. For $0 < \beta < 1$, the scores from both branches are combined using Eq. (21).

Fig. 11 plots the performance changes across five datasets for various β values. (1) *Performance changes:* As β increases, the model performance initially increases, then decreases, and finally increases again. This indicates that combining the anomaly scores from both branches is beneficial but requires some focus. (2) *Focusing on univariate datasets:* The model performance initially increases slightly, then declines. It remains relatively stable within the range of [0, 0.25]. (3) *Considering multivariate datasets:* The detection accuracy fluctuates less on multivariate datasets, remaining stability within the range of [0.7, 1.0]. (4) *In summary:* The ranges [0, 0.25] and [0.7, 1.0] are stable, where the LTFAD model performs optimally.

4.9.2. MLP layers (L) and the dimension of each layer (d)

The number of MLP layers (L) and the dimension of each layer (d) are another two important parameters that directly affect the timeliness and resource consumption of LTFAD. The smaller L and d, the lighter the LTFAD is, with fewer parameters, worse representation ability, less computing and storage resource usage, and higher timeliness. Conversely, the bigger L and d, the heavier the LTFAD is, with more parameters, stronger representation ability, more computing and storage resource usage, and lower timeliness.

Fig. 12 shows the effect of two parameters on the performance. (1) *MLP Layers:* The performance first rises and then falls with increasing L. It is not difficult to find that LTFAD can balance lightweight and high-accuracy when $L = 2$. (2) *Parameter d:* When L is fixed at 2, the model performs optimally with d in the range of 78 to 150. (3) *In summary:* A two-layer MLP with a 128-dimensional hidden layer is a optimal choice for our lightweight LTFAD.

4.10. Discussion

This paper aims to pursue timestamp-level anomaly detection for IIoT time series with high speed, high accuracy and low resource consumption. Extensive experiments show that the LTFAD model achieves the expected three goals.

• *High speed:* The timeliness and deployability experiments show that LTFAD significantly needs less detection time compared to SOTA deep TSAD models. Roughly speaking, the detection time of LTFAD is 4–10 times faster than other deep models. This advantage is attributed to the fact that the LTFAD model is built on a shallow All-MLP lightweight architecture. In addition, the learning process of LTFAD on M variables and two branches is performed in parallel.

•*High accuracy:* The accuracy, robustness, ablation, and sensitivity experiments demonstrate that four metrics(ACC, Precision, Recall and F-Score) of the LTFAD model rank first or second on five univariate and seven multivariate datasets, except the Recall score on Dodgers, the Precision score on SVDB, the Precision score on MSL and the Precision score on SWaT. Moreover, on the SVDB dataset, the F-Score score of LTFAD is 4.7% higher than that of the second-ranked baseline. These results are attributed to task- specific accuracy enhancement strategies: "local to global reconstruction learning", "global to local reconstruction learning" and "time–frequency joint learning".

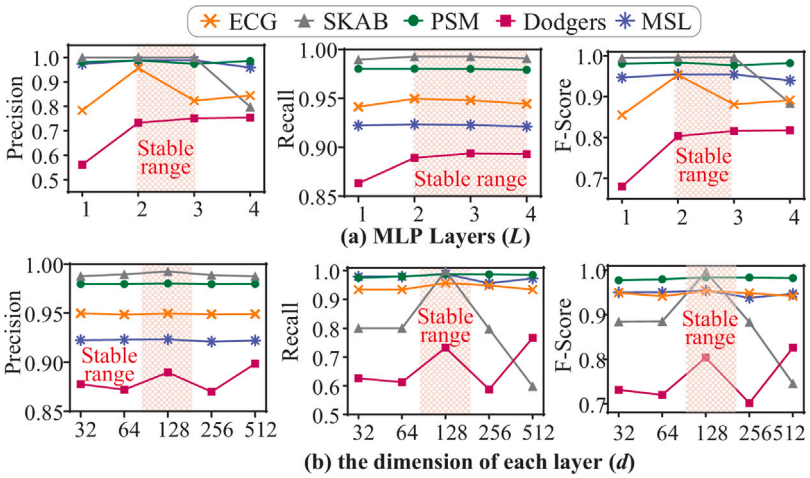


Fig. 12. Sensitivity for two key parameters, where (a) is L and (b) is d.

•*Low resource consumption:* Due to its shallow and lightweight All-MLP architecture, LTFAD exhibits significantly lower resource consumption than other SOTA deep models. Specifically, in terms of model parameter, the parameters of LTFAD are only 1/3 to 1/10 of other deep models. On the Raspberry Pi 4b, the CPU-Usage of LTFAD is 25% to 200% lower than that of other deep models. This efficiency makes it can be deployed on resource-constrained devices such as the Raspberry Pi 4b, which has only 2 G of RAM.

5. Conclusion

In this paper, we develop LTFAD, a lightweight All-MLP time–frequency anomaly detection model for IIoT time series. Unlike traditional deep and bulky neural network-based solutions, LTFAD employs only four parallelized two-layer MLPs to build a lightweight dual-branch reconstruction network with high timeliness and low resource consumption. To improve model accuracy, LTFAD designs two reconstruction branches: "local to global reconstruction learning" and "global to local reconstruction learning". To further enhance accuracy, time–frequency joint learning is employed in each reconstruction branch. To the best of our knowledge, this is the first work to develop a time–frequency anomaly detection model only based on the shallow All-MLP architecture. Comparative evaluations on twelve IIoT time series and three edge devices demonstrate the superior performance of LTFAD.

Note that, there are some limitations in the proposed LTFAD. Firstly, the advantages of the LTFAD model cannot be directly reproduced in other time series analysis tasks, such as forecasting and classification. Specifically, the advantage of LTFAD comes from the shallow All-MLP architecture and task-specific accuracy enhancement strategies. The shallow All-MLP architecture provides high timeliness and low resource consumption, while task-specific accuracy enhancement strategies improve the model accuracy. Although the shallow All-MLP architecture can be migrated to other analysis tasks, the accuracy enhancement strategies cannot be directly copied. Secondly, the performance of LTFAD is sensitive to parameter β on some datasets.

In the future, two research works will be further explored to address these limitations. One work is to design a universal time-series analysis model for multiple analysis tasks based only on a shallow All-MLP architecture. The other work is to develop a lightweight optimization mechanism for LTFAD to automatically select appropriate parameters for different datasets.

CRediT authorship contribution statement

Lei Chen: Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Software, Resources, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Data curation, Conceptualization. **Xinzhe Cao:** Validation, Software, Methodology, Formal analysis, Conceptualization. **Tingqin He:** Writing – review & editing, Supervision, Project administration, Investigation, Data curation. **Yepeng Xu:** Writing – review & editing, Visualization, Validation, Software, Project administration, Data curation. **Xuxin Liu:** Writing – review & editing, Visualization, Investigation, Data curation. **Bowen hu:** Writing – review & editing, Validation, Investigation, Data curation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This research was funded by the National Natural Science Foundation of China (No. 62103143); the Hunan Provincial Natural Science Foundation of China (No. 2024JJ5162); the Young Backbone Teacher of Hunan Province (No. 2022101), and the Scientific Research Fund of Hunan Provincial Education Department (No. 22B0471).

Data availability

I have shared the link to my data/code in the manuscript.

References

Audibert, J., Michiardi, P., Guyard, F., Marti, S., & Zuluaga, M. A. (2022). Do deep neural networks contribute to multivariate time series anomaly detection? *Pattern Recognition*, *132*, Article 108945.

Carletti, M., Terzi, M., & Susto, G. A. (2023). Interpretable anomaly detection with diffi: Depth-based feature importance of isolation forest. *Engineering Applications of Artificial Intelligence*, *119*, Article 105730.

Chang, Y., Chen, J., Su, R., Xie, J., & Li, A. (2024). Two-phase dual-adversarial agents with multivariate information for unsupervised anomaly detection of IIoT-edge devices. *IEEE Internet of Things Journal*, *11*(13), 23577–23591.

Chen, S.-A., Li, C.-L., Arik, S. O., Yoder, N. C., & Pfister, T. (2023). TSMixer: An all-MLP architecture for time series forecasting. *Transactions on Machine Learning Research*.

Chen, Y., Xu, H., Pang, G., Qiao, H., Zhou, Y., & Shang, M. (2024). Self-supervised spatial-temporal normality learning for time series anomaly detection. arXiv preprint arXiv:2406.19770.

Ding, C., Sun, S., & Zhao, J. (2023). MST-GAT: A multimodal spatial–temporal graph attention network for time series anomaly detection. *Information Fusion*, *89*, 527–536.

Duan, Z., Xu, H., Wang, Y., Huang, Y., Ren, A., Xu, Z., et al. (2022). Multivariate time-series classification with hierarchical variational graph pooling. *Neural Networks*, *154*, 481–490.

Fan, J., Liu, Z., Wu, H., Wu, J., Si, Z., Hao, P., et al. (2023). LUAD: A lightweight unsupervised anomaly detection scheme for multivariate time series data. *Neurocomputing*, *557*, Article 126644.

Fan, J., Wang, Z., Wu, H., Sun, D., Wu, J., & Lu, X. (2023). An adversarial time–frequency reconstruction network for unsupervised anomaly detection. *Neural Networks*, *168*, 44–56.

Fang, Y., Xie, J., Zhao, Y., Chen, L., Gao, Y., & Zheng, K. (2024). Temporal-frequency masked autoencoders for time series anomaly detection. In *2024 IEEE 40th international conference on data engineering* (pp. 1228–1241).

Gong, W., Wang, Y., Zhang, M., Mihankhah, E., Chen, H., & Wang, D. (2022). A fast anomaly diagnosis approach based on modified CNN and multisensor data fusion. *IEEE Transactions on Industrial Electronics*, *69*(12), 13636–13646.

He, S., Guo, M., Li, Z., Lei, Y., Zhou, S., Xie, K., et al. (2023). A joint matrix factorization and clustering scheme for irregular time series data. *Information Sciences*, *644*, Article 119220.

He, S., Guo, Q., Li, G., Xie, K., & Sharma, P. K. (2025). Multivariate time series anomaly detection based on multiple spatiotemporal graph convolution. *IEEE Transactions on Instrumentation and Measurement*, *74*, 1–14.

He, S., Li, Z., Wang, J., & Xiong, N. N. (2021). Intelligent detection for key performance indicators in industrial-based cyber-physical systems. *IEEE Transactions on Industrial Informatics*, *17*(8), 5799–5809.

He, S., Li, G., Xie, K., & Sharma, P. K. (2024). Fusion graph structure learning-based multivariate time series anomaly detection with structured prior knowledge. *IEEE Transactions on Information Forensics and Security*, *19*, 8760–8772.

He, S., Li, G., Yi, T., Alfarraj, O., Tolba, A., Kumar Sangaiah, A., et al. (2024). Graph structure learning-based multivariate time series anomaly detection in internet of things for human-centric consumer applications. *IEEE Transactions on Consumer Electronics*, *70*(3), 5419–5431.

Huang, S., Liu, Y., Cui, H., Zhang, F., Li, J., Zhang, X., et al. (2024). MEAformer: An all-MLP transformer with temporal external attention for long-term time series forecasting. *Information Sciences*, *669*, Article 120605.

Langfu, C., Zhang, Q., Yan, S., Liman, Y., Yixuan, W., Junle, W., et al. (2023). A method for satellite time series anomaly detection based on fast-DTW and improved-KNN. *Chinese Journal of Aeronautics*, *36*(2), 149–159.

Li, J., Izakian, H., Pedrycz, W., & Jamal, I. (2021). Clustering-based anomaly detection in multivariate time series data. *Applied Soft Computing*, *100*, Article 106919.

Li, X., Xie, K., Wang, X., Xie, G., Li, K., & Cao, J. (2023). A light-weight and robust tensor convolutional autoencoder for anomaly detection. *IEEE Transactions on Knowledge and Data Engineering*, 1–14.

Liu, Y., Ju, B., Yang, D., Peng, L., Li, D., Sun, P., et al. (2024). Memory-enhanced spatial-temporal encoding framework for industrial anomaly detection system. *Expert Systems with Applications*, *250*, Article 123718.

Nam, Y., Yoon, S., Shin, Y., Bae, M., Song, H., Lee, J.-G., et al. (2024). Breaking the time-frequency granularity discrepancy in time-series anomaly detection. In *Proceedings of the ACM on web conference 2024* (pp. 4204–4215).

Nizam, H., Zafar, S., Lv, Z., Wang, F., & Hu, X. (2022). Real-time deep anomaly detection framework for multivariate time-series data in industrial iot. *IEEE Sensors Journal*, *22*(23), 22836–22849.

Paparrizos, J., Boniol, P., Palpanas, T., Tsay, R. S., Elmore, A., & Franklin, M. J. (2022). Volume under the surface: a new accuracy evaluation measure for time-series anomaly detection. *Proceedings of the VLDB Endowment*, *15*(11), 2774–2787.

Pevnỳ, T. (2016). Loda: Lightweight on-line detector of anomalies. *Machine Learning*, *102*, 275–304.

Sgueglia, A., Di Sorbo, A., Visaggio, C. A., & Canfora, G. (2022). A systematic literature review of IoT time series anomaly detection solutions. *Future Generation Computer Systems*, *134*, 170–186.

Sivapalan, G., Nundy, K. K., Dev, S., Cardiff, B., & John, D. (2022). ANNet: A lightweight neural network for ECG anomaly detection in IoT edge sensors. *IEEE Transactions on Biomedical Circuits and Systems*, *16*(1), 24–35.

Tang, C., Xu, L., Yang, B., Tang, Y., & Zhao, D. (2023). GRU-based interpretable multivariate time series anomaly detection in industrial control system. *Computers and Security*, *127*, Article 103094.

Tian, Z., Zhuo, M., Liu, L., Chen, J., & Zhou, S. (2023). Anomaly detection using spatial and temporal information in multivariate time series. *Scientific Reports*, *13*(1), 4400.

Truong, H. T., Ta, B. P., Le, Q. A., Nguyen, D. M., Le, C. T., Nguyen, H. X., et al. (2022). Light-weight federated learning-based anomaly detection for time-series data in industrial control systems. *Computers in Industry*, *140*, Article 103692.

Tu, F.-F., Liu, D.-J., Yan, Z.-W., Jin, X.-B., & Geng, G.-G. (2024). STFT-TCAN: A TCN-attention based multivariate time series anomaly detection architecture with time-frequency analysis for cyber-industrial systems. *Computers and Security*, Article 103961.

Wang, Z., Pei, C., Ma, M., Wang, X., Li, Z., Pei, D., et al. (2024). Revisiting VAE for unsupervised time series anomaly detection: A frequency perspective. (pp. 3096–3105). New York, NY, USA: Association for Computing Machinery.

Wang, Z., Ruan, S., Huang, T., Zhou, H., Zhang, S., Wang, Y., et al. (2024). A lightweight multi-layer perceptron for efficient multivariate time series forecasting. *Knowledge-Based Systems*, *288*, Article 111463.

Xu, H., Pang, G., Wang, Y., & Wang, Y. (2023). Deep isolation forest for anomaly detection. *IEEE Transactions on Knowledge and Data Engineering*, *35*(12), 12591–12604.

Xu, H., Wang, Y., Jian, S., Liao, Q., & Wang, Y. (2024). Calibrated one-class classification for unsupervised time series anomaly detection. *IEEE Transactions on Knowledge and Data Engineering*, 1–14.

Xu, J., Wu, H., Wang, J., & Long, M. (2022). Anomaly transformer: Time series anomaly detection with association discrepancy. In *International conference on learning representations*.

Xu, Z., Zeng, A., & Xu, Q. (2024). FITS: Modeling time series with 10k parameters. In *The twelfth international conference on learning representations*.

Yang, Y., Zhang, C., Zhou, T., Wen, Q., & Sun, L. (2023). Dcdetector: Dual attention contrastive representation learning for time series anomaly detection. In *Proceedings of the 29th ACM SIGKDD conference on knowledge discovery and data mining* (pp. 3033–3045).

Yao, Y., Ma, J., & Ye, Y. (2022). KfreqGAN: Unsupervised detection of sequence anomaly with adversarial learning and frequency domain information. *Knowledge-Based Systems*, *236*, Article 107757.

Yi, K., Zhang, Q., Fan, W., Wang, S., Wang, P., He, H., et al. (2024). Frequency-domain MLPs are more effective learners in time series forecasting. *Advances in Neural Information Processing Systems*, *36*.

Yu, J., Gao, X., Li, B., Zhai, F., Lu, J., Xue, B., et al. (2024). A filter-augmented auto-encoder with learnable normalization for robust multivariate time series anomaly detection. *Neural Networks*, *170*, 478–493.

rui Yu, L., hong Lu, Q., & Xue, Y. (2024). DTAAD: Dual Tcn-attention networks for anomaly detection in multivariate time series data. *Knowledge-Based Systems*, *295*, Article 111849.

Zhang, Z., Guo, D., Zhou, S., Zhang, J., & Lin, Y. (2023). Flight trajectory prediction enabled by time-frequency wavelet transform. *Nature Communications*, *14*(1), 5258.

Zhang, X., Zhao, Z., Tsiligkaridis, T., & Zitnik, M. (2022). Self-supervised contrastive pre-training for time series via time-frequency consistency. *Advances in Neural Information Processing Systems*, *35*, 3988–4003.

Zhang, C., Zhou, T., Wen, Q., & Sun, L. (2022). TFAD: A decomposition time series anomaly detection architecture with time-frequency analysis. In *Proceedings of the 31st ACM international conference on information and knowledge management* (pp. 2497–2507).

Zheng, Y., Koh, H. Y., Jin, M., Chi, L., Wang, H., Phan, K. T., et al. (2024). Graph spatiotemporal process for multivariate time series anomaly detection with missing values. *Information Fusion*, *106*, Article 102255.

Zhong, Z., Yu, Z., Xi, X., Xu, Y., Chen, J., & Yang, K. (2024). SimAD: A simple dissimilarity-based approach for time series anomaly detection. arXiv preprint arXiv:2405.11238.

Zhong, Z., Yu, Z., Yang, Y., Wang, W., & Yang, K. (2024). PatchAD: Patch-based MLP-mixer for time series anomaly detection. arXiv preprint arXiv:2401.09793.

Zhou, X., Wu, J., Liang, W., Wang, K. I.-K., Yan, Z., Yang, L. T., et al. (2024). Reconstructed graph neural network with knowledge distillation for lightweight anomaly detection. *IEEE Transactions on Neural Networks and Learning Systems*, 1–12.

Zhu, W., Li, W., Dorsey, E. R., & Luo, J. (2023). Unsupervised anomaly detection by densely contrastive learning for time series data. *Neural Networks*, *168*, 450–458.